

UNIVERSITÀ DEGLI STUDI DI VERONA

CORSO DI LAUREA IN INFORMATICA

Reti di Calcolatori

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1	Introduzione alle reti	4
1.1	Comunicazione tra elaboratori	4
1.2	Commutazione di circuito e di pacchetto	7
1.3	Classificazione e misura dei ritardi	8
1.4	Modelli di riferimento ISO-OSI e TCP/IP	11
1.5	Indirizzi IP e subnetting	14
1.6	Esercizi svolti	17
2	Livello applicazione	21
2.1	Il modello client/server	21
2.2	Hyper Text Transfer Protocol - HTTP	22
2.3	Domain Name Service - DNS	26
2.4	Posta elettronica - SMTP e IMAP	28
3	Livello trasporto	30
3.1	Scopi e servizi	30
3.2	Protocollo TCP	31
3.3	Protocollo UDP	38
3.4	Esercizi svolti	39
4	Livello network	43
4.1	Protocollo IP	43
4.2	Algoritmi di routing	45
4.3	Protocolli DHCP e ICMP	49
4.4	Network Address Translation e IPv6	50
4.5	Esercizi svolti	54
5	Livello di collegamento dati	56
5.1	Servizi e accesso al canale condiviso	56
5.2	Algoritmi di accesso al canale condiviso	57
5.3	Ethernet, switch e LAN, Bridge	64

<i>INDICE</i>	3
5.4 Wireless LAN	67
5.5 Esercizi svolti	70

Capitolo 1

Introduzione alle reti

1.1 Comunicazione tra elaboratori

Il corso di reti di calcolatori si pone come obiettivo quello di risolvere il problema della **comunicazione tra computer**, inteso come il trasferimento di informazioni tra due o più sistemi, studiato in questo corso con un approccio **top-down**, ovvero a partire da un alto livello per andare sempre più a basso livello. In particolare, si presentano i due problemi fondamentali della comunicazione:

- **Protocollo di comunicazione:** Insieme di regole logiche che definiscono come le entità devono comunicare, nello specifico il **formato** dei messaggi, il loro **ordine** e le **azioni** da compiere alla loro ricezione.
- **Trasporto delle informazioni:** Il trasferimento fisico di dati fra due dispositivi. Richiede un'infrastruttura di rete composta da mezzi trasmissivi, come cavi o fibra ottica, dispositivi di rete e, naturalmente, protocolli di trasmissione.

Possiamo paragonare lo scambio di informazioni tra elaboratori con l'invio di una lettera. Il mittente scrive il messaggio, inserisce il foglio in una busta, con scritto l'indirizzo del destinatario e la consegna all'ufficio postale. Quest'ultimo la trasporterà fino al destinatario descritto, il quale potrà aprire la busta e leggere il contenuto del messaggio. Affinché il processo funzioni correttamente, è necessario seguire regole precise (quindi seguire il protocollo), come scrivere correttamente l'indirizzo del destinatario.

Nella pratica, la comunicazione tra due calcolatori avviene tramite una sequenza di livelli, spesso rappresentata come una pila verticale, dove ogni livello svolge una funzione specifica e prepara i dati per quello successivo. Intuitivamente:

1. L'applicazione genera il messaggio.
2. Il messaggio è preparato per la trasmissione.

3. I dati vengono inviati tramite la rete.
4. Il destinatario riceve i dati.
5. I dati sono elaborati fino a raggiungere l'applicazione finale.

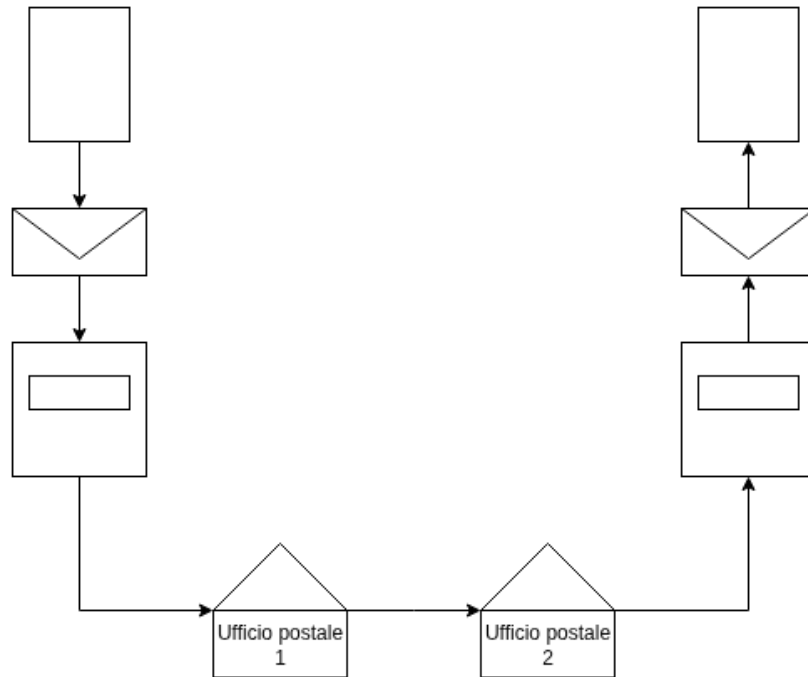


Figura 1.1: Esempio di comunicazione intuitivo

Naturalmente, lo scambio di informazioni non è unidirezionale, infatti si dice che il processo di trasferimento informazioni è **simmetrico** tra mittente e destinatario. Definiamo ora la struttura delle architetture di rete per capire in che modo comunichino gli elaboratori: possono avere elementi diversi, ma tutte presentano necessariamente gli **end-host**, ovvero i dispositivi per l'invio e ricezione di informazioni come computer o smartphone, gli **apparati di rete**, che permettono l'instradamento o la distribuzione dei dati nella rete, come router, switch e access point, e i **collegamenti**, i mezzi fisici o wireless che permettono la trasmissione dei dati, come fibra ottica o Wi-Fi.

Possiamo descrivere **Internet** in termini di infrastruttura di rete su scala globale, la quale fornisce servizi ad applicazioni distribuite. Per carpirne il funzionamento analizziamo un esempio su scala minore: le reti **Local Area Network** (LAN). Queste sono reti che coprono un'area limitata, come ad esempio un ufficio o una casa, dove sono presenti gli end-host. I principali dispositivi utilizzati sono:

- **Switch**: Dispositivo che collega fra loro più connessioni cablate.

- **Access point:** Dispositivo wireless che consente agli end-host di collegarsi alla rete.

Tipicamente lo switch rappresenta il punto centrale della rete cablata, mentre l'access point vi si collega per fornire connettività wireless. Nelle reti domestiche moderne è comune che router, switch e access point siano integrati nello stesso apparato.

Le LAN sono collegate alla rete Internet grazie ad un **router di bordo**, il quale, oltre al suddetto compito, è capace di inoltrare i dati ad altri router della rete globale, detta **backbone**, fino al raggiungimento di quello di destinazione, che consegnerà i dati all'end-host.

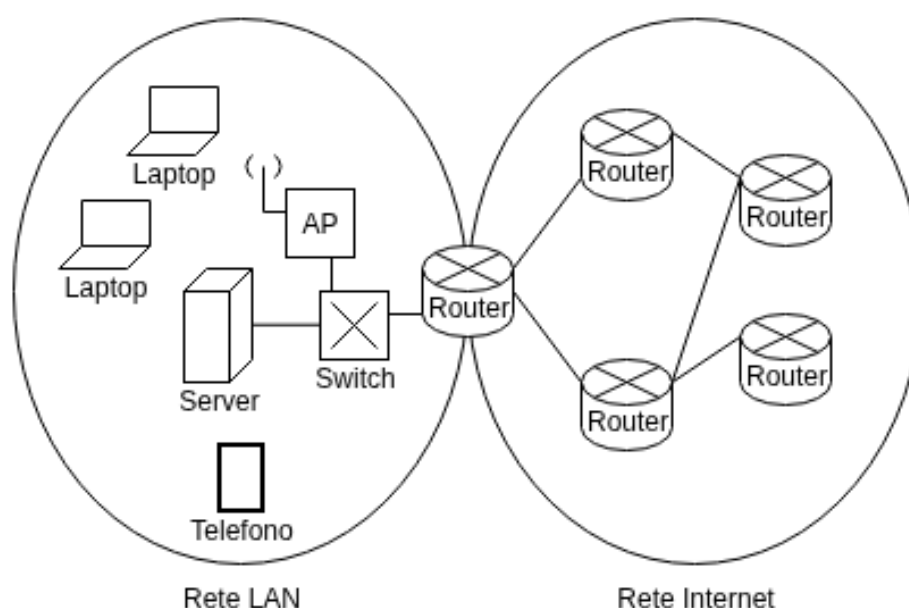


Figura 1.2: Collegamento tra rete LAN e Internet

La connettività Internet è fornita da organizzazioni chiamate **Internet Service Provider** (ISP), divise in tre livelli gerarchici:

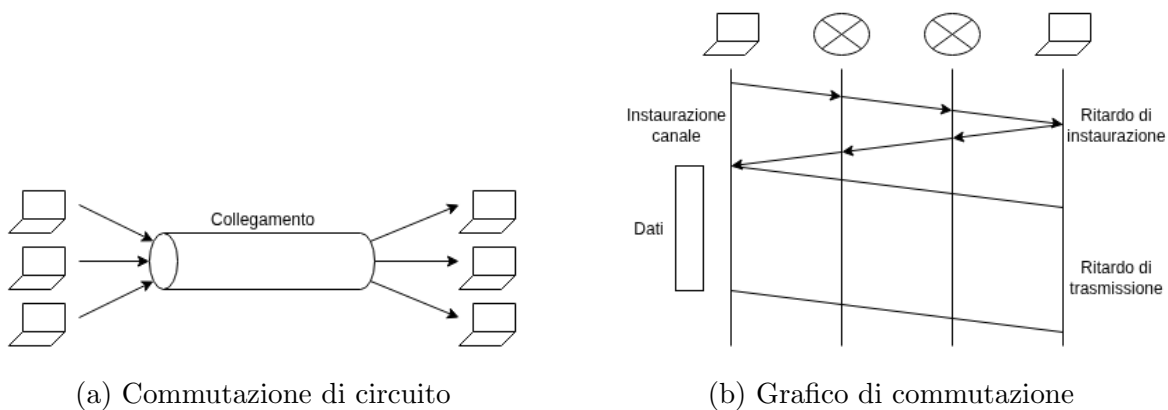
- **Tier 1:** Operatori globali che gestiscono infrastrutture backbone internazionali.
- **Tier 2:** Operatori nazionali o regionali che acquistano connettività dai Tier-1 ma possono anche stabilire collegamenti diretti tra loro.
- **Tier 3:** Operatori locali che forniscono connettività agli utenti finali, come abitazioni, aziende, hotel.

In particolare, esistono gli **Autonomous Systems** (AS); insiemi di reti gestite da un'unica entità amministrativa e con una politica di routing comune.

1.2 Commutazione di circuito e di pacchetto

Ci sono due modalità principali per il trasferimento dei dati fra calcolatori: la **commutazione di circuito** e di **pacchetto**. La prima è stata storicamente usata nelle reti telefoniche tradizionali e ad oggi è nettamente meno diffusa. Il percorso tra la sorgente e la destinazione è allocato staticamente per la durata dello scambio dell'informazione, vedendo quindi le risorse di trasmissione dedicate alla comunicazione. Tuttavia, essendo la banda limitata, si può sostenere solo un certo numero di circuiti allo stesso tempo.

La gestione delle risorse avviene tramite il **Time Division Multiplexing**, una divisione delle risorse in slot da fornire ai vari utenti. Alternativamente si utilizza il **Frequency Division Multiplexing**, una divisione di spazio sulle frequenze sicché gli slot non interferiscano gli uni con gli altri, seguendo la logica della radio.

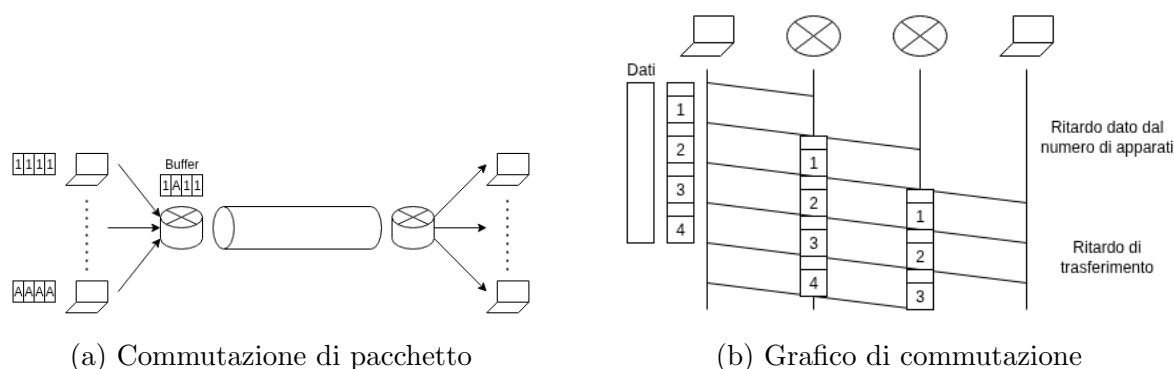


Questo sistema di commutazione ha due pro: **risorse dedicate** a ciascun utente, creando una comunicazione senza interruzioni o accodamento, e il **ritardo prevedibile** e costante dopo l'instaurazione del circuito. I problemi riguardano in primo luogo la **banda limitata**, poiché dando slot dedicati, in caso fossero tutti occupati, non sarebbe possibile usare il servizio. In secondo luogo parliamo di spreco delle risorse, perché, fin tanto che rimane aperto, il canale è mantenuto attivo anche quando non si trasferiscono i dati.

La **commutazione di pacchetto** invece vede i dati da trasferire divisi in sezioni indipendenti, a loro volta divise in due parti: l'**header**, contenente informazioni di controllo, tra cui indirizzo sorgente e destinazione, e la seconda, contenente i dati del pacchetto. Le varie sezioni sono inviate al router, il quale le accoda con un buffer per inviarle una alla volta passando per un bottleneck. Con questa metodologia è possibile gestire un numero molto elevato di utenti, poiché le risorse sono condivise dinamicamente. Infatti, non ci sono sprechi di risorse in caso un utente smetta di trasferire dati.

Il contro di questa modalità di commutazione è invece l'introduzione dei ritardi. I pacchetti si possono trasferire solo quando sono stati ricevuti tutti i relativi bit contenenti

le informazioni tramite la modalità **store-and-forward switching**¹. In particolare, è presente un ritardo iniziale legato al numero di apparati attraversati; smaltito questo, ci sarà quello di trasmissione. Un secondo problema è dato dalla potenziale **perdita di pacchetti**, perché in quanto i buffer hanno spazio limitato, può capitare di richiedere il trasferimento di pacchetti quando questo è pieno. In tal caso, il pacchetto viene scartato.



1.3 Classificazione e misura dei ritardi

Le reti di calcolatori sono caratterizzate da un **throughput** limitato, inteso come la quantità di dati al secondo che può essere trasferita tra due sistemi; introducono ritardi e si possono perdere anche pacchetti. Sebbene questo non sia uno scenario ideale, possiamo analizzare i diversi tipi di ritardo e cercare di usare gli strumenti a disposizione per affrontare il problema. Generalmente, le prestazioni delle applicazioni per internet sono pesantemente influenzate dai ritardi di rete, somma dei ritardi introdotti in ciascun nodo, dati da ogni hop di router e composti da:

- **Ritardo di elaborazione:** Legato all'architettura del router, è il tempo impiegato per esaminare l'intestazione del pacchetto e determinare dove dirigerlo. In questo

¹Informazione salvata nel router, poi inoltrata a quello di destinazione.

intervallo di tempo si possono anche controllare errori a livello di bit avvenuti nella trasmissione. Dopo l'elaborazione, si invia il pacchetto verso la coda che precede il collegamento al prossimo router.

- **Ritardo di accodamento:** Inserito in coda, il pacchetto deve attendere la trasmissione sul collegamento. Il tempo di attesa è legato al numero di pacchetti precedentemente arrivati. Infatti, nelle politiche first-come-first-served, comunemente in uso nelle reti a commutazione di pacchetto, il pacchetto può essere trasferito solamente dopo la trasmissione di tutti quelli che lo precedono nell'arrivo. Normalmente è il ritardo che incide più di tutti.
- **Ritardo di trasmissione:** Riguarda il tempo necessario per immettere tutti i bit del pacchetto nel canale di trasmissione. È un valore che dipende dalla sua dimensione e dalla velocità di trasmissione del router. In particolare:

$$d_{trans} = L/R$$

Con L dimensione del pacchetto in bit e R velocità di trasmissione in bit al secondo.

- **Ritardo di propagazione:** Immesso nel collegamento, questo è il tempo utilizzato per la propagazione di un bit fino al prossimo router. È quindi un valore legato dal mezzo fisico per la trasmissione e la distanza fra i due router. In particolare:

$$d_{prop} = d/v$$

Con d distanza tra i due router e v la velocità di propagazione nel collegamento.

Nella pratica, il ritardo complessivo dipende anche dal contesto geografico. Generalmente, valgono i seguenti valori:

- **Comunicazione su stessa LAN:** $< 1ms$
- **Comunicazione con stesso ISP:** $5 - 20ms$
- **Comunicazione su stesso continente:** $20 - 50ms$
- **Comunicazione intercontinentale:** $100 - 200ms$

Abbiamo poi a disposizione alcuni strumenti per la misurazione dei ritardi; il primo è il **ping**, un tool a CLI che permette di misurare il **Round Trip Time**, ovvero il tempo di andata e ritorno di un messaggio fra due elaboratori. Prende in input il nome logico utilizzato dal sito, come "www.google.com", e stampa a video indirizzo IP dell'host (approfondito in sezioni successive), minimo, media, massimo e deviazione standard del roundtrip time.

```
f27@F27-Laptop:~$ ping www.google.it
PING www.google.it (142.251.209.35) 56(84) bytes of data:
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=1 ttl=117 time=50.6 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=2 ttl=117 time=55.4 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=3 ttl=117 time=63.3 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=4 ttl=117 time=68.0 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=5 ttl=117 time=61.9 ms
^C
--- www.google.it ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 50.594/59.829/68.018/6.129 ms
```

Figura 1.5: Esempio di ping

Un altro strumento è il **traceroute**, il quale prende in input il nome dell'host come per il ping. Questo comando consente di vedere il percorso intrapreso dai pacchetti (tracciare il percorso, difatti) e determinare i ritardi di andata e ritorno per ogni singolo hop effettuato. In particolare, il sistema dell'utente invia un certo numero di pacchetti verso la destinazione e ad ogni router attraversato, questi rimandano un messaggio all'origine contenente il loro nome e indirizzo IP. Può capitare inoltre:

- **Router nascosto:** Se l'hop è indicato con tre asterischi, gli amministratori di rete hanno celato le sue informazioni.
- **Cammini equivalenti:** Se più indirizzi IP sono indicati nel passo, significa che gli hop sono equivalenti per raggiungere una stessa destinazione.

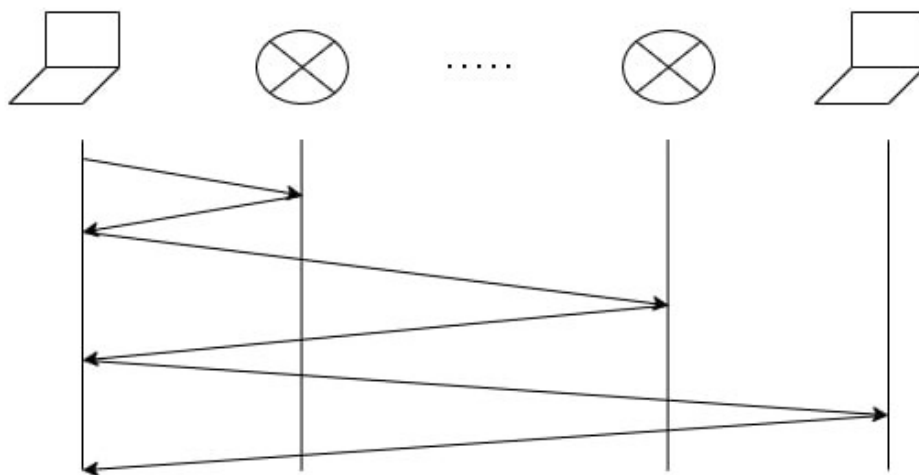


Figura 1.6: Visualizzazione di traceroute

Un'altra misura importante per le prestazioni di una rete è il **throughput end-to-end**. Si tratta, in ogni istante di tempo, della velocità alla quale un'entità sta ricevendo un determinato insieme di informazioni, come per esempio un file. Si tratta di un valore misurato in *bps* (bit per second). Generalmente si misura anche il **throughput medio**, dato dal rapporto fra il numero di bit F e il tempo T impiegato per la loro intera ricezione: $\bar{R} = F/T$.

Capiamo dunque che è un valore dipendente dalla velocità di trasmissione. Non a caso, essendo i link collegati tramite dei **bottleneck**, la velocità di trasferimento è determinata dal link con capacità minima, risultando anche quello più lento.

1.4 Modelli di riferimento ISO-OSI e TCP/IP

La modalità di trasferimento delle informazioni è organizzata in base al **modello a strati**. Il problema della comunicazione fra due elaboratori ha una parte che riguarda la logica, quindi la gestione delle informazioni, e una che riguarda la fisica, ovvero la strutturazione dell'architettura di rete. L'approccio utilizzato per la risoluzione del primo problema è dato dal **Divide et Impera**: intuitivamente, date due entità comunicanti, la prima "impacchetta" il messaggio aggiungendo un header per ogni strato, e la seconda lo "spacchetta" rimuovendone uno per livello per poterlo leggere.

In un modello simile potremmo aggiungere un numero indefinito di livelli il cui insieme è detto **stack protocollare** ed è imperativo che vengano eseguiti con un ordine specifico, perché se così non fosse, il messaggio potrebbe non essere compreso.

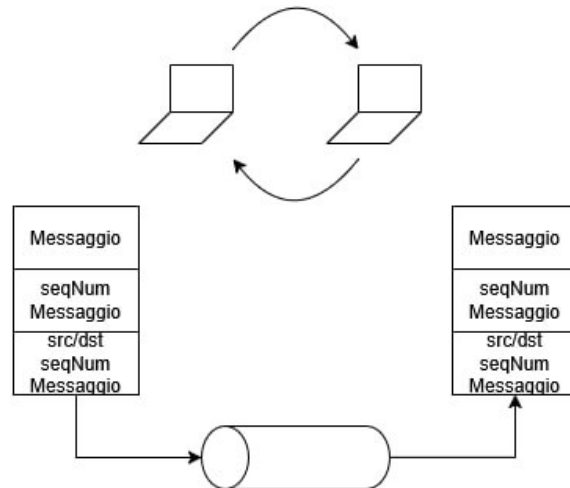


Figura 1.7: Modello a strati

Per esempio, se un calcolatore volesse inviare un messaggio, si potrebbe aggiungere prima un header per salvare il numero di sequenza del pacchetto. Essendo questi inviati non

necessariamente con un ordine fisso, risulterebbe utile conoscere l'ordine per ricomporli. In uno strato inferiore poi si potrebbero aggiungere i dati per indicare l'origine e la destinazione del messaggio. Inviando questo prodotto al secondo calcolatore, inizierebbe col rimuovere l'intestazione degli indirizzi, poi quella dei numeri di sequenza, per poi leggere il messaggio ordinato.

Un primo modello che utilizza questa struttura è l'**ISO/OSI**; proposto dalla International Organization for Standardization, per la Open System Interconnection. Si tratta del primo riferimento per la descrizione delle operazioni eseguite in ogni livello.

Il modello identifica due entità: gli **end-system**, i calcolatori, e gli **intermediate-systems**, i router. Entrambe condividono i primi tre livelli a partire dalla base: quello **fisico** per la gestione dei bit, quello di **collegamento dati** per determinare la destinazione e quello di **rete**, responsabile dell'instradamento. Gli end-system hanno inoltre quattro livelli aggiuntivi: **trasporto**, **sessione**, **presentazione** e **applicazione**.

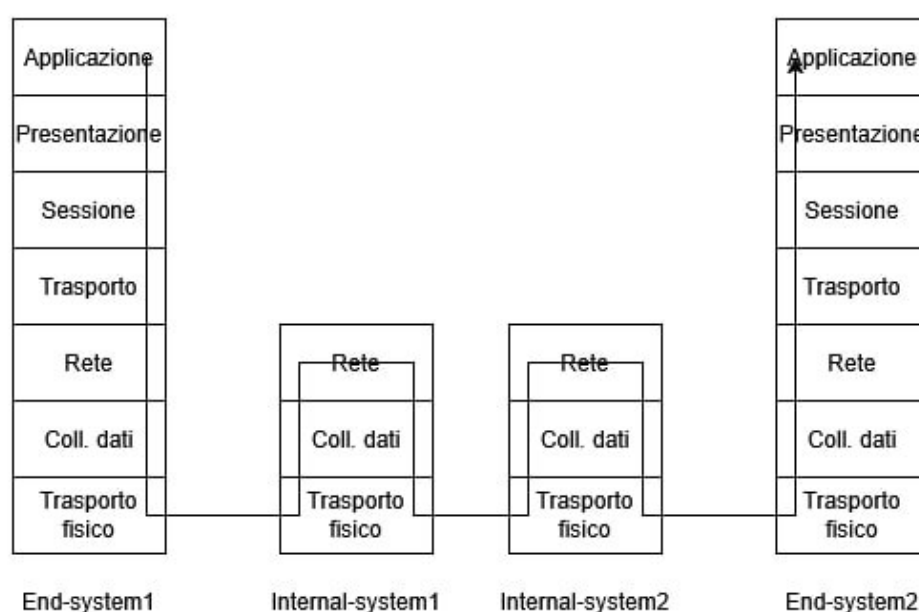


Figura 1.8: Modello ISO/OSI

Nella pratica si usa un modello che usa questo come base, ma presenta solo i livelli aggiuntivi di trasporto e applicazione. Si chiama **modello TCP/IP** ed il nome deriva dai due principali protocolli usati in questi due livelli: **Transmission Control Protocol** e **Internet Protocol**. Analizziamone ogni strato:

- **Livello di applicazione**: Sede delle applicazioni di rete e dei relativi protocolli come HTTP (scambio documenti web), SMTP (scambio mail), FTP (scambio

file tra due sistemi remoti). Generalmente scambia pacchetti di informazioni, i **messaggi**, con l'applicazione in un altro sistema periferico.

- **Livello di trasporto:** Si occupa del trasferimento dei messaggi tra punti periferici gestiti dalle applicazioni. Si usa il protocollo **TCP**, che fornisce un servizio orientato alla connessione, quindi che permette di stabilirla, usarla e rilasciarla. Include inoltre la garanzia di consegna dei messaggi a livello applicazione e un controllo di flusso, ottenuto frazionando i messaggi lunghi in più **segmenti**.
- **Livello di rete:** Si occupa di trasferire i pacchetti a livello di rete, detti **datagrammi**, da un host a un altro. Mette a disposizione il servizio di consegna segmento al livello di trasporto nell'host ricettivo. Comprende in primo luogo il protocollo **IP**, che definisce i campi dei datagrammi e come i sistemi agiscono su di essi. In secondo luogo ha vari protocolli di instradamento che determinano i percorsi da intraprendere.
- **Livello di collegamento:** Fornisce servizi per il trasferimento dei datagrammi fra i nodi del percorso, i quali dipendono dal protocollo utilizzato. È verosimile che questi vengano gestiti da protocolli diversi come Wi-Fi o Ethernet. A questo livello i pacchetti sono chiamati **frame**.
- **Livello fisico:** Si occupa del trasporto dei singoli bit dei frame da un nodo a quello successivo. Le modalità dipendono dal protocollo attuato, ma anche dal mezzo trasmissivo, come fibra ottica o doppino.

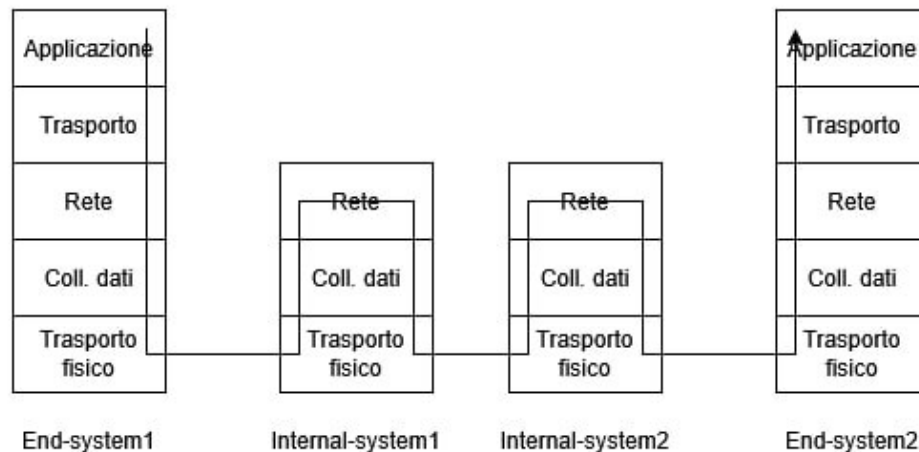


Figura 1.9: Modello TCP/IP

Questo processo che attraversa ogni strato è detto **incapsulamento**. Ora che conosciamo il senso degli strati, procediamo a definire le entità coinvolte nella comunicazione: a

livello applicativo, questa avviene tra processi in esecuzione sugli end-host. Affinché due processi possano comunicare, è necessario identificarli univocamente.

Un processo è identificato tramite un **numero di porta**, definito a livello di trasporto, mentre l'host sul quale è in esecuzione si identifica con un **indirizzo IP**. L'insieme di indirizzo IP e numero di porta costituisce un **endpoint di comunicazione**. Formalmente, una comunicazione tra due processi è identificata dalla seguente tupla:

$$(\text{IP}_A, \text{IP}_B, \text{porta}_A, \text{porta}_B)$$

1.5 Indirizzi IP e subnetting

Gli **indirizzi IP**, o indirizzi di protocollo internet, sono identificativi univoci di un'interfaccia di rete di un host o router all'interno della rete Internet. Sebbene si tenda ad associare questo codice di quattro numeri ad una sola macchina, è possibile, tramite alcune strategie, usarne uno diverso.

Naturalmente, i siti web non sono letti con sequenze di numeri dalle persone, ragion per cui presentano un grado di astrazione dato dal **nome di dominio**, come per esempio "www.google.com", al quale è associato l'indirizzo IP. La traduzione fra i due avviene a livello di applicazione tramite il **sistema DNS** (Domain Name System), il cui prodotto sarà usato a livello rete per aggiungere un'intestazione contenente l'indirizzo di sorgente e quello di destinazione. In base alla versione del protocollo abbiamo diversi bit totali per contenerli: 32b per **IPv4** e 128b per **IPv6**.

Gli indirizzi IP sono letti come sequenze di numeri in base binaria. Per esempio, con protocollo IPv4, il sistema potrebbe leggere:

$$1001\ 1101\ 0001\ 1011\ 0001\ 0011\ 0111\ 1011$$

Per una scrittura più contratta si utilizza la **notazione decimale puntata**, dove si considerano blocchi di 8b e si fa il cambio di base in decimale. Usando i numeri appena visti otteniamo:

$$\begin{aligned} 10011101 &= 157 \\ 00011011 &= 27 \\ 00010011 &= 19 \\ 01111011 &= 123 \end{aligned} \tag{1.1}$$

Dunque il nostro indirizzo IP si annovera con 157.27.19.123. Da questi valori è possibile inoltre ricavare alcune utili informazioni. Non dissimilmente dai numeri telefonici per le reti fisse, ogni numero ha un significato. Prendendo come esempio +39 045 802 7059, sappiamo che +39 indica l'area geografica internazionale, 045 la provincia di Verona, 802

l'organizzazione specifica alla quale è associato il numero, e 7059 il suffisso, che indica il numero di interfaccia in uso.

La stessa logica vale per gli indirizzi IP: in questo esempio i primi tre blocchi sono l'identificativo di una rete specifica e il loro insieme è noto come **prefisso**, mentre l'ultimo blocco è detto **suffisso** e indica l'interfaccia dell'host. Ciò significa che possiamo ricondurci ad un dispositivo preciso in base al suo IP.

Nella notazione puntata, è possibile che un'organizzazione abbia un elevato numero di dispositivi, rendendo fuorviante la comprensione del suffisso. Per eludere questo problema, si indica il numero di bit che compongono il prefisso con una barra: 157.27.19.123/16. In particolare, esistono dei valori particolari riservati ad utilizzi specifici e non per l'identificazione dell'host:

- **This Host:** Un indirizzo in cui ogni bit è posto a zero, ovvero 0.0.0.0. Può essere anche usato per gli indirizzi non assegnati.
- **Local Broadcast:** Un indirizzo in cui ogni bit è posto a uno. Per il protocollo IPv4 vale 255.255.255.255.
- **Indirizzo di rete:** Un indirizzo il cui suffisso ha tutti i bit posti a zero, mentre la rete è indicata dal prefisso, come 127.27.0.0.
- **Directed Broadcast:** Un indirizzo in cui il suffisso ha tutti i bit posti a uno, mentre la rete è indicata dal prefisso, come 127.27.255.255.

Per identificare gli host disponibili in una determinata rete, i calcolatori utilizzano la **netmask**; una sequenza di 32b associata ad un indirizzo IP in cui i bit del prefisso sono posti a uno, mentre gli altri a zero. Possiamo annotarla, come già visto, in notazione decimale puntata o esadecimale. Sulla maschera, i calcolatori effettuano un'operazione di AND bit a bit con l'indirizzo IP, ottenendo l'indirizzo di rete.

È possibile verificare i valori di indirizzo IP e netmask con un comando a linea da terminale; per i sistemi UNIX è chiamato **ifconfig**², ma se si vuole andare più a fondo, usando il comando **whois** seguito da un indirizzo IP, è possibile ottenere informazioni sull'assegnazione degli indirizzi, il nome della rete e anche l'ISP al quale è allacciata.

Tra l'altro, questi due comandi possono presentare valori il cui prefisso ha un numero di bit diverso; ciò può accadere a causa della scelta architetturale di **subnetting**: la suddivisione di una rete principale in varie sottoreti, le quali avranno indirizzi ad esse collegate. Supponiamo infatti di voler dividere una rete principale di indirizzo 157.27.0.0/16 in due sottoreti di dimensione equivalente. Si ritiene necessario avere un modo per distinguere a quale delle due siamo connessi. A partire dall'indirizzo IP, attacchiamo un bit del suffisso al prefisso per usarlo come discriminante. Ciò ci fa ottenere:

²Il comando è stato deprecato di recente. Consigliabile usare **ip addr**.

- subNet1: prefisso(1011 1101 0001 1011 0), suffisso(000 0000 0000), notazione decimale: 157.27.0.0/17
- subNet2: prefisso(1101 1101 0001 1011 1), suffisso(000 0000 0000), notazione decimale: 157.27.128.0/17

Questa scelta riduce il numero di host disponibili per ciascuna sottorete, ma consente una gestione più efficiente della rete. Inoltre, questa struttura è impercettibile dall'esterno, in quanto i router di bordo delle sottoreti si connettono a quello della rete intera.

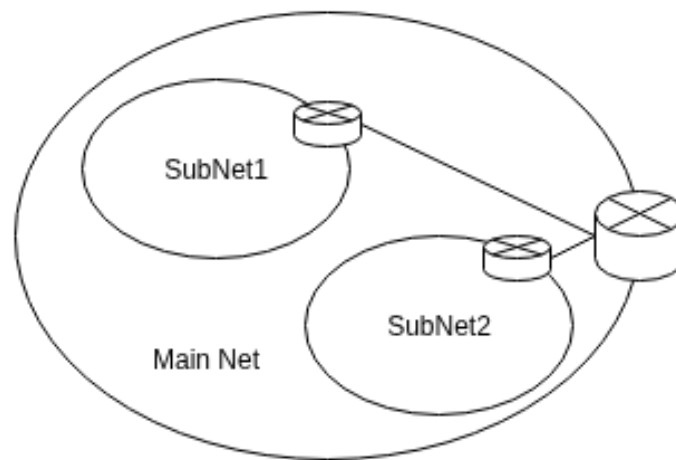


Figura 1.10: Subnetting

I blocchi di indirizzi sono detti **CIDR**: Classless Inter-Domain Router, sui quali è necessario aprire una parentesi storica. In precedenza, infatti, si chiamavano Classful ed essendo al tempo un servizio non ancora utilizzato su larga scala, ci si poteva permettere di "sprecare" disponibilità. Il numero di bit dedicati al prefisso era infatti predeterminato e codificato nello stesso; veniva usato per distinguere le diverse classi. In particolare:

- **Classe A**: L'indirizzo ha 0 come prima cifra e il prefisso ha $8b$ di dimensione. Numero di reti: 2^7 . Numero di host per rete massimi: 2^{24} .
- **Classe B**: L'indirizzo ha 10 come prime due cifre e il prefisso ha $16b$ di dimensione. Numero di reti: 2^{14} . Numero di host per rete massimi: 2^{16} .
- **Classe C**: L'indirizzo ha 110 come prime tre cifre ed il prefisso ha $24b$ di dimensione. Numero di reti: 2^{21} . Numero di host per rete massimi: 2^8 .

1.6 Esercizi svolti

Esercizio 1: Dato l'indirizzo IP 140.120.84.20/20, calcolare l'indirizzo di rete.

L'indirizzo di rete si ricava a partire da quello fornito, ponendo tutti i bit del suffisso a zero. Convertiamo l'IP in binario:

$$140.120.84.20/20 \implies 10001100.01111000.0101/0100.00010100$$

Tutti i bit del suffisso devono essere posti a zero. Ritraduciamo il risultato in notazione decimale puntata:

$$10001100.01111000.0101/0000.00000000 \implies 140.120.80.0/20 \checkmark$$

Esercizio 2: Dato il blocco di indirizzi 165.5.1.0/24 creare tre sottoreti per tre LAN in tal modo che tutte e tre abbiano lo stesso numero totale di indirizzi.

Per creare sottoreti è necessario sacrificare bit del suffisso; questi saranno usati come discriminante per distinguere le LAN. Convertiamo anzitutto in binario l'indirizzo di rete:

$$165.5.1.0/24 \implies 10100101.00000101.00000001./00000000$$

Per tre reti è necessario utilizzare due bit di suffisso, poiché possono rappresentare al massimo quattro combinazioni. Dato l'indirizzo di rete binario, otteniamo quindi:

- LAN1: 10100101.00000101.00000001.00/000000 $\implies 165.5.1.0/26 \checkmark$
- LAN2: 10100101.00000101.00000001.01/000000 $\implies 165.5.1.64/26 \checkmark$
- LAN3: 10100101.00000101.00000001.10/000000 $\implies 165.5.1.128/26 \checkmark$

Esercizio 3: Dato il blocco di indirizzi del precedente esercizio, 165.5.1.0/24 creare tre sottoreti per tre LAN tali che LAN1 contenga il doppio degli indirizzi rispetto alle altre due.

La soluzione è presto detta. Dividiamo inizialmente in due il blocco principale e assegniamo una delle due parti a LAN1, per poi dividere nuovamente l'altra e assegnare questi blocchi a LAN2 e LAN3:

- LAN1: 10100101.00000101.00000001.0/0000000 $\implies 165.5.1.0/25 \checkmark$
- LAN2: 10100101.00000101.00000001.10/0000000 $\implies 165.5.1.128/26 \checkmark$
- LAN3: 10100101.00000101.00000001.11/0000000 $\implies 165.5.1.192/26 \checkmark$

Esercizio 4: Supponiamo una rete; due indirizzi assegnati a due host ad essa connessi sono 101.75.79.255 e 101.75.80.0. Qual è il blocco CIDR più piccolo che li contiene?

Per ricavare il prefisso è necessario confrontare i bit dei due indirizzi. Tutti quelli che coincidono formano il prefisso:

- Host1: 101.75.79.255 $\Rightarrow$ 01100101.01001011.01001111.11111111
- Host2: 101.75.80.0 $\Rightarrow$ 01100101.01001011.01010000.00000000

Notiamo che i bit combaciano fino al /19, quindi l'indirizzo di rete è dato da:

$$01100101.01001011.01010000.00000000 \Rightarrow 101.75.64.0/19 \checkmark$$

A partire da questo blocco si vogliono creare cinque sottoreti con i seguenti vincoli:

- LAN1: Prefisso di /21
- LAN2: Deve ospitare almeno 1000 host.
- LAN3: Prefisso di /23
- LAN4: Deve ospitare almeno 400 host.
- LAN5: Deve avere a disposizione metà degli indirizzi del blocco di partenza.

Qui aiuta molto ragionare a partire dalle sezioni più grandi. Il blocco che abbiamo trovato ha $2^{13} = 8192$ indirizzi disponibili da spartire tra le sottoreti. Quella che ne richiede di più tra tutte è LAN5, la quale necessita la metà del blocco. Sacrifichiamo un bit:

$$LAN5 = 01100101.01001011.0100/0000.00000000 \Rightarrow 101.75.64.0/20$$

Rimangono a disposizione $2^{12} = 4096$ indirizzi. Notiamo che LAN1 richiede /21 bit di prefisso, quindi suddividiamo in due il blocco e assegnamole una parte:

$$LAN1 = 01100101.01001011.01010/000.00000000 \Rightarrow 101.75.80.0/21$$

Rimangono a disposizione $2^{11} = 2048$ indirizzi. Possiamo garantire a LAN2 i suoi 1024 host suddividendo ancora il blocco:

$$LAN2 = 01100101.01001011.010110/00.00000000 \Rightarrow 101.75.88.0/22$$

I rimanenti $2^{10} = 1024$ indirizzi si possono spartire senza problemi tra LAN3 e LAN4, siccome richiedono la stessa cosa in termini diversi. Il risultato finale del subnetting è quindi:

- LAN1: 01100101.01001011.01010/000.00000000 $\Rightarrow$ 101.75.80.0/21 ✓
- LAN2: 01100101.01001011.010110/00.00000000 $\Rightarrow$ 101.75.88.0/22 ✓
- LAN3: 01100101.01001011.0101110/0.00000000 $\Rightarrow$ 101.75.92.0/23 ✓
- LAN4: 01100101.01001011.0101111/0.00000000 $\Rightarrow$ 101.75.94.0/23 ✓
- LAN5: 01100101.01001011.0100/0000.00000000 $\Rightarrow$ 101.75.64.0/20 ✓

Esercizio 5: Supponiamo una rete da dividere in tre sottoreti, le quali devono rispettare i seguenti vincoli:

- LAN1: Deve contenere almeno 300 indirizzi.
- LAN2: Deve contenere almeno 40 indirizzi.
- LAN3: Deve contenere almeno 90 indirizzi.

Trovare il blocco CIDR minimo che le contiene, sprecando meno indirizzi possibile e sapendo che l'indirizzo broadcast di LAN3 è 148.12.79.255

Per prima cosa, è necessario tradurre i vincoli in potenze di 2, per capire l'effettivo totale di indirizzi e quindi bit di suffisso necessari per la realizzazione del subnetting:

$$300, 40, 90 \Rightarrow 512, 64, 128 \Rightarrow 1024b$$

Il blocco avrà quindi /22 bit di prefisso. Ricaviamolo a partire da quello di broadcast e convertiamolo in binario:

$$148.12.79.255/22 \Rightarrow 10010100.00001100.010011/11.11111111$$

Ricavando l'indirizzo di rete ponendo i bit del suffisso a zero, sarà possibile ottenere le seguenti sottoreti, ridotte al limite per evitare sprechi di indirizzi:

- LAN1: 10010100.00001100.0100110/0.00000000 $\Rightarrow$ 148.12.76.0/23 ✓
- LAN3: 10110100.00001100.01001111.1/00000000 $\Rightarrow$ 148.12.79.128/25 ✓
- LAN2: 10010100.00001100.01001111.01/00000000 $\Rightarrow$ 148.12.79.64/26 ✓

Esercizio 6: Sia una blocco di rete con il quale si vuole creare quattro sottoreti con i seguenti vincoli:

- LAN1: Richiede almeno 130 host. Contiene inoltre l'indirizzo 46.144.141.41
- LAN2: Richiede almeno 270 host.

- LAN3: Richiede almeno 65 host.
- LAN4: Richiede almeno 35 host.

Ricavare il blocco CIDR minimo che le contenga sprecaendo meno indirizzi possibile.

Uguale al procedimento del precedente esercizio. Sommando le potenze del 2 più vicine alla richiesta, otteniamo la necessità di 1024 indirizzi minimo e quindi un prefisso del CIDR di /22 bits. Traduciamo l'indirizzo di rete dall'host fornito:

$$46.144.141.41/22 \Rightarrow 00101110.10100000.100011/00.00000000 \Rightarrow 46.144.140.0/22$$

Suddividiamo dunque il blocco a partire dalle richieste più grandi a quelle più piccole. L'ordine sarà quindi di soddisfare LAN2, LAN1, LAN3 e infine LAN4, ottenendo:

- LAN2: $00101110.10100000.100011/0.00000000 \Rightarrow 46.144.142.0/23 \checkmark$
- LAN1: $00101110.10100000.10001101./00000000 \Rightarrow 46.144.141.0/24 \checkmark$
- LAN3: $00101110.10100000.10001100.1/00000000 \Rightarrow 46.144.140.128/25 \checkmark$
- LAN4: $00101110.10100000.10001100.01/00000000 \Rightarrow 46.144.140.64/26 \checkmark$

Esercizio 7: Sia un blocco di rete con il quale si vuole creare tre sottoreti con i seguenti vincoli:

- LAN1: Richiede almeno 400 host. Contiene inoltre l'indirizzo 178.242.85.168.
- LAN2: Richiede almeno 300 host.
- LAN3: Richiede almeno 1200 host.

Ricavare il blocco CIDR minimo che le contenga sprecaendo meno indirizzi possibile.

Uguale al procedimento del precedente esercizio. Non mi ripeterò. $400 \Rightarrow 512$, $300 \Rightarrow 512$, $1200 \Rightarrow 2048$. Per contenere questo totale, avremo bisogno di $2^{12} = 4096$ indirizzi; il prefisso avrà 20b, ricaviamone il blocco CIDR:

$$178.242.85.168/20 \Rightarrow 10110010.11110010.0101/0000.00000000 \Rightarrow 178.242.80.0/20$$

Suddividiamo il blocco in base alle richieste. È possibile usare una notazione più contratta che mischia notazione decimale puntata con quella in binario:

- LAN3: $178.242.01011/000.00000000 \Rightarrow 178.242.88.0/21 \checkmark$
- LAN2: $178.242.0101011/0.00000000 \Rightarrow 178.242.86.0/23 \checkmark$
- LAN1: $178.242.0101010/0.00000000 \Rightarrow 178.242.84.0/23 \checkmark$

Capitolo 2

Livello applicazione

2.1 Il modello client/server

Al livello applicativo sono generati i messaggi da scambiare tra i calcolatori, come per esempio richieste e risposte HTTP, un protocollo associato ai browser web, messaggi di posta elettronica, o porzioni di video. I messaggi generati dall'applicazione vengono passati al livello di trasporto, che può suddividerli in segmenti e aggiungere intestazioni contenenti informazioni di controllo come il numero di sequenza. Le sezioni devono poi selezionare il servizio del livello di trasporto, tra:

- **Protocollo TCP:** Servizio orientato alla connessione il quale garantisce la consegna dell'informazione.
- **Protocollo UDP:** Servizio non orientato alla connessione il quale non garantisce la consegna dell'informazione.

Applicazioni diverse hanno necessità di servizi diversi. Per esempio, un client di posta elettronica richiede che ogni informazione sia consegnata al destinatario; di conseguenza userà un protocollo che si appoggia alla garanzia di consegna del TCP, come **SMTP**.

Invece, un sito web come Twitch fornisce un contenuto "Tollerante alle perdite", quindi se non tutti i dati sono consegnati non necessariamente l'esperienza sarà danneggiata. Per esempio durante una livestream è richiesto di mantenere una connessione in tempo reale, privilegiando la continuità della riproduzione rispetto alla completa affidabilità dei dati, accettando eventuali ritardi o perdite limitate.

La "selezione del servizio" per l'invio di un messaggio si traduce in termini di apertura di un **socket** verso la destinazione, ovvero un'astrazione software che rappresenta il punto di accesso tra applicazione e livello di trasporto. Per aprirne uno è necessario specificare **protocollo** di trasporto, **indirizzi IP** di sorgente e destinazione e **porta** di sorgente e destinazione, le quali identificano l'interfaccia. I due processi che comunicano con il socket si identificano nei ruoli di:

- **Client:** Processo che inizia la comunicazione.
- **Server:** Il processo in attesa di contatto; accetta le richieste di comunicazione.

È un errore comune quello di confondere l'host con il client. Sebbene spesso un host coincida con un singolo client o server, una macchina può eseguire contemporaneamente più processi, assumendo quindi entrambi i ruoli.

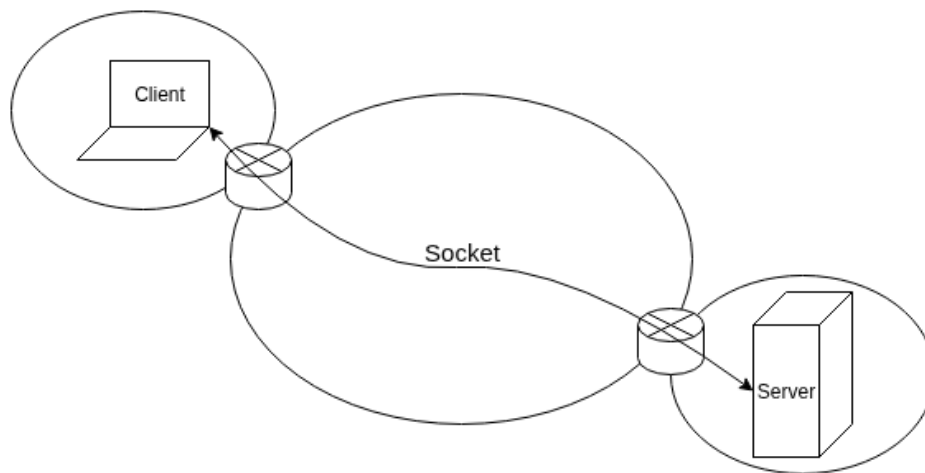


Figura 2.1: Apertura di un socket

2.2 Hyper Text Transfer Protocol - HTTP

HTTP è un protocollo a livello di applicazione del Web implementato in due processi client e server, i quali comunicano mediante uno scambio di messaggi HTTP. Il protocollo definisce struttura e modalità di scambio dei messaggi e si serve di **pagine Web**, generalmente costituite da un file HTML (Hypertext Markup Language) principale detto **index.html** entro il quale si riferenziano gli **oggetti**; file indirizzabili tramite URL come immagini, file JS o un foglio di stile CSS.

Il lato client del protocollo è implementato da un **browser Web**, come Firefox, per l'invio delle richieste, mentre il server è gestito dal **Web server**, il quale ospita gli oggetti menzionati prima. Quando un utente richiede una pagina Web, il browser invia al server messaggi di **richiesta HTTP** per ottenere gli oggetti nella pagina; si riceve quindi una **risposta HTTP**, la quale può essere positiva, confermando che il trasferimento degli oggetti è andato a buon fine, oppure negativa se la pagina non è stata trovata o è inesistente.

HTTP si appoggia al protocollo di trasporto TCP; infatti, il client stabilisce in primo luogo una connessione TCP con il server, alla quale i processi accedono tramite le proprie

socket. Inoltre, il server invia i file richiesti senza memorizzare alcuna informazione di stato del client; per questa ragione diciamo che HTTP è un protocollo **senza memoria di stato**.

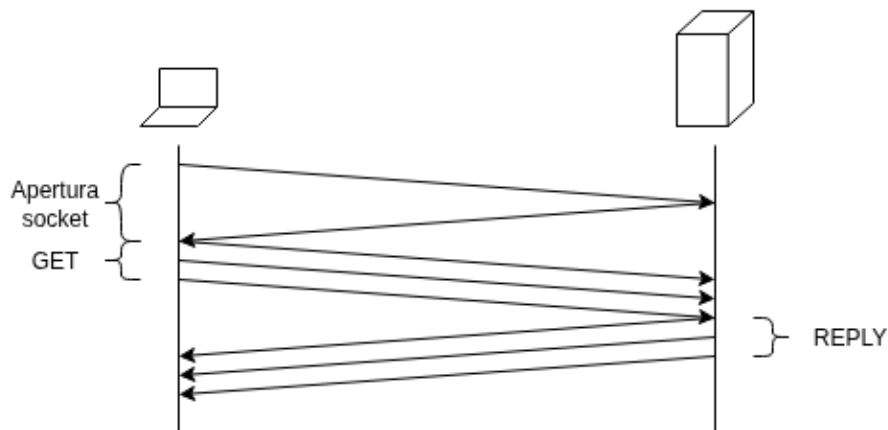


Figura 2.2: Protocollo HTTP

Presentiamo ora la struttura dei messaggi di richiesta e risposta HTTP. Il primo è composto da più righe: la prima è detta **riga di richiesta** e presenta tre campi:

- **Metodo:** Indica la procedura da attuare, come GET per richiedere risorse, POST per inviare informazioni al server, o DELETE, per cancellare informazioni sul server.
- **URL:** Identifica gli oggetti sui quali usare il metodo.
- **Versione HTTP:** Indica la versione del protocollo utilizzata.

Le righe successive sono dette di **intestazione** e tutte tranne quella che indica l'host (nella versione 1.1) sono opzionali. Determinano dati diversi come il browser utilizzato o la lingua del messaggio da inviare, come anche il tipo di connessione. Segue esempio:

```
GET /index.html HTTP/1.1
Host: www.univr.it
Connection: close
User-agent: Mozilla/4.0
Accept-language: it
```

I messaggi di risposta HTTP presentano invece tre sezioni: una **riga di stato** iniziale, delle **righe di intestazione** e il **corpo**, contenente la risorsa richiesta.

La prima riga è composta a sua volta da tre campi: la versione del protocollo, il codice di stato e il suo corrispettivo messaggio di stato, come [200-OK], [400-Bad request] o [404-Not found]. Le successive righe invece indicano molte informazioni come, tra l'altro, il tipo di connessione, la data e l'ora di ottenimento e invio della risorsa, il nome del web server e la data e l'ora dell'ultima modifica della risorsa. Segue esempio:

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2020 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2020 15:11:03 GMT
Content-Length: 3164
Content-Type: text/html
(Contenuto della risorsa...)
```

Nella creazione delle applicazioni web gli sviluppatori devono fare una scelta: inviare le coppie richiesta/risposta su connessioni TCP separate o sulla stessa? Il primo approccio si realizza con **connessioni non persistenti**, mentre il secondo con le **persistenti**. Sono entrambe scelte valide e funzionali, sebbene di default, HTTP implementi la seconda modalità.

Supponiamo di avere una connessione non persistente e che un client richieda le risorse di una pagina web con 10 immagini JPEG. In sequenza, avviene:

1. Client HTTP inizializza una connessione TCP con il server sulla porta 80, default per HTTP. Associate alla connessione ci sono due socket: uno per il client, l'altro per il server.
2. Il Client HTTP invia tramite il socket una richiesta HTTP al server.
3. Il Server HTTP riceve il messaggio del client, recupera l'oggetto dalla propria memoria e lo incapsula in una risposta HTTP per inviargliela tramite il socket.
4. Il Server HTTP comunica a TCP di terminare la connessione, cosa che avviene una volta consegnato il messaggio intero al client.
5. Il Client riceve la risposta, estrae il file dal messaggio e lo esamina, trovando i riferimenti alle immagini JPEG.
6. I primi quattro passi sono ripetuti per ogni oggetto da recuperare.

Essendo che si apre e chiude una connessione TCP per ogni risorsa da recuperare, avremo un totale di 11 connessioni. Queste possono essere settate per lavorare sequenzialmente o in parallelo.

A partire da HTTP 1.1 sono state introdotte le connessioni persistenti, dove il server mantiene la connessione TCP aperta anche dopo l'invio di tutte le risorse. Si possono inviare intere pagine web in una volta, come anche più pagine web, allo stesso client. La connessione ha termine quando rimane inattiva per un dato lasso di tempo configurabile.

Come già menzionato, i server HTTP sono privi di stato, per rendere lo sviluppo di applicazioni più semplice e con prestazioni migliori. Tuttavia, accade spesso di dover autenticare gli utenti, per ragioni di sicurezza e per fornire contenuti in funzione della loro identità. I **cookies** servono esattamente a questi scopi: consentono ai server di tener conto degli utenti.

Al primo utilizzo della pagina web, il server crea un codice associato all'utente che verrà inviato nella reply HTTP con l'header **SET COOKIE**. Il client, se accetta, memorizzerà l'indirizzo web e lo assocerà al codice ricevuto. Nelle successive interazioni, se i cookie sono stati accettati, all'invio della GET da parte del client sarà presente anche un campo con il codice del cookie, il quale verrà elaborato dal server per fornire una risposta ritagliata per l'utente, a prescindere dall'indirizzo IP con il quale si collega.

Un'altra importante entità di rete è la **web cache**, nota anche come proxy server. Si pone nel percorso fra client e web server e soddisfa richieste HTTP al posto di quest'ultimo con lo scopo di aumentare le prestazioni. Diminuisce infatti il ritardo di accesso alle risorse più utilizzate e di consegna dei messaggi; come conseguenza diretta, il carico di rete è nettamente diminuito.

Senza web cache, avremmo una situazione nella quale più utenti in una rete richiedono una stessa risorsa, più si sperimenterà un ritardo. Se invece l'architettura di rete la comprende, si presenterebbe il seguente scenario:

1. Browser stabilisce una connessione TCP col proxy server e invia una richiesta HTTP per la risorsa.
2. Il proxy controlla la presenza di una copia della risorsa all'interno della propria memoria. Se è rilevata, la ritorna al client, altrimenti si apre una connessione TCP con il web server di origine per ottenerla.
3. Quando il proxy riceve l'oggetto, ne salva una copia in memoria e ne invia un'altra al browser web.

In genere le cache riescono a soddisfare il 70-80% delle richieste, ma usando un server diverso da quello originale si pone il problema dell'attualizzazione dei dati. Se una risorsa è modificata nel server originale, questa non è automaticamente inviata al proxy. La soluzione alla questione si presenta nella forma del **get condizionale**: una richiesta get-HTTP con una riga di intestazione **if-modified-since**. Fondamentalmente, il browser effettua la richiesta specificando una data, ed è ricevuta dal proxy. Questo comunicherà

con il web server che ritornerà la risorsa se è stata modificata entro la data specificata all'inizio. Se non ha subito modifiche, il proxy può inviarlo direttamente, altrimenti la risorsa deve essere ripresa dal web server originale.

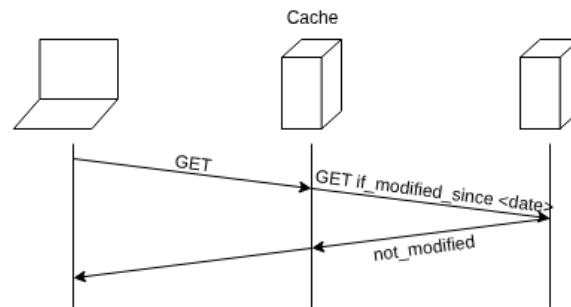


Figura 2.3: Web caching

2.3 Domain Name Service - DNS

Il servizio di directory di Internet **Domain Name Service** (DNS) ha lo scopo primario di tradurre i nomi di dominio associati ad un indirizzo IP per ritornare quest'ultimo e renderlo leggibile ai calcolatori. Si tratta in primo luogo di un database di server distribuito a livello mondiale implementato con una logica gerarchica e in secundis è un protocollo a livello di applicazione che consente agli host di interrogare la base di dati. In particolare, poggia su UDP e usa la porta 53; viene spesso utilizzato da altri protocolli a livello di applicazione, come HTTP e SMTP.

Supponiamo adesso che un web browser richieda la traduzione del nome di dominio "www.google.it". Il funzionamento del DNS segue le seguenti fasi:

1. Il calcolatore dell'utente esegue il lato client dell'applicazione DNS.
2. Il browser estrae il nome dell'host dall'URL e lo passa al lato client dell'applicazione DNS.
3. Il client DNS invia una **DNS query** contenente questo nome al server DNS.
4. Il server DNS ritorna l'indirizzo IP corrispondente al dominio ricevuto.
5. Ricevuto l'indirizzo IP, il browser web può adesso instaurare una connessione TCP verso il processo server collegato alla porta 80 di quell'indirizzo IP.

Come si può intuire, questa procedura introduce un ritardo, ma è comune che l'indirizzo IP desiderato si trovi all'interno di server DNS cache vicini, con lo scopo di ridurre il traffico in rete e il ritardo del servizio.

Il DNS offre anche altri servizi, come quello di **host** e **mail aliasing**, la possibilità di usare un insieme di nomi logici dal valore equivalente (sinonimi) per ricondursi ad uno stesso host, e la **distribuzione del carico di rete**, perché è possibile replicare su più web server uno stesso nome di dominio particolarmente richiesto, al quale è associato un insieme di indirizzi IP; quando richiesto dalla query, il DNS server risponderà con l'intero insieme di indirizzi variando l'ordinamento a ogni risposta.

Come già detto, più server DNS sono distribuiti per il mondo attraverso un **sistema gerarchico**; supponendo che un web browser effettui una DNS query, si esplorerebbero nel seguente ordine le tipologie dei server DNS:

1. **Server radice:** Contengono un insieme di indirizzi dei server TLD.
2. **Server Top-level domain:** Si occupano dei domini di primo livello come com, it, org e simili. Forniscono gli indirizzi IP dei server autoritativi.
3. **Server DNS autoritativi:** Appartenenti alle organizzazioni, specificano il loro dominio, come google, univr e simili.

Infine, le organizzazioni posseggono dei **server DNS locali**, i cui indirizzi IP si possono ottenere tramite una richiesta di protocollo DHCP.

La distribuzione dei DNS server risolve in particolare il problema del sovraccarico, distribuendo il traffico di query, e quello della disponibilità del servizio, poiché in caso di mancata risposta da parte di un server è possibile interrogarne un altro dello stesso livello.

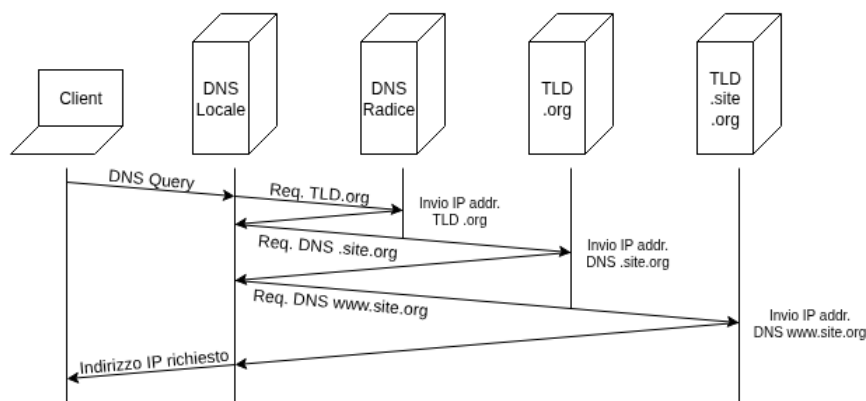


Figura 2.4: Ricerca del DNS

2.4 Posta elettronica - SMTP e IMAP

La posta elettronica è un mezzo di comunicazione asincrono che consente agli utenti di inviare e ricevere messaggi testuali; le e-mail moderne presentano inoltre altri oggetti, come **allegati**, **collegamenti ipertestuali**, **foto incorporate** e testo con **formattazione HTML**. Il meccanismo funziona grazie a tre elementi principali:

- **User agent:** Conosciuti anche come mail client, consentono di leggere, rispondere, inoltrare, salvare e comporre i messaggi. Esempi pratici sono Outlook, Gmail o Tutamail.
- **Mail server:** Costituiscono la parte centrale dell'infrastruttura del servizio; una casella di posta contiene i messaggi ricevuti alla quale è possibile accedere tramite autenticazione. Presenta inoltre una coda dei messaggi dove si trattengono quelli non ancora inviati.
- **Protocollo SMTP:** Il principale protocollo a livello di applicazione su Internet; si affida al TCP per trasferire i messaggi.

Lo scambio di messaggi tra due utenti vede generalmente una comunicazione diretta tra i due mail server; ciò significa che non sono utilizzati server intermedi, e se quello del destinatario è spento, si riproverà a inviare la mail. In condizioni ideali, invece, segue sei fasi:

1. L'utente A invoca il proprio user agent e fornisce l'indirizzo di posta elettronica dell'utente B. Compose il messaggio e dà istruzione allo user agent di inviarlo.
2. Lo user agent dell'utente A invia il messaggio al suo mail server, dove è collocato in una coda di messaggi.
3. Il lato client di SMTP, eseguito sul server dell'utente A, vede il messaggio in coda e apre una connessione TCP sulla porta 25 verso un server SMTP in esecuzione sul mail server dell'utente B.
4. Dopo un SMTP-handshake, il client SMTP invia il messaggio dell'utente A sulla connessione TCP instaurata.
5. Presso il mail server dell'utente B, il lato server di SMTP riceve il messaggio e lo posiziona nella casella postale.
6. Quando ritenuto opportuno, l'utente B potrà aprire e leggere la mail invocando il proprio user agent che userà il relativo protocollo di lettura. Ad oggi è spesso usato IMAP, altrimenti sono diffuse interfacce user agent basate su HTTP.

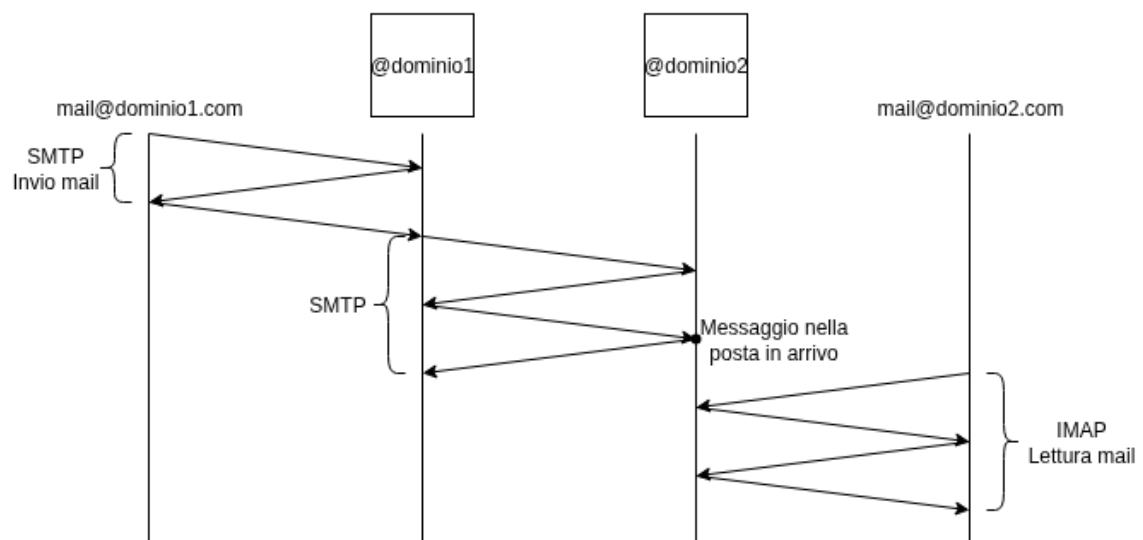


Figura 2.5: Invio e ricezione posta elettronica

Capitolo 3

Livello trasporto

3.1 Scopi e servizi

I protocolli a livello di trasporto mettono a disposizione una **comunicazione logica** tra processi applicativi di due host diversi. Sono implementati nei sistemi periferici; lato client, il livello di trasporto converte i messaggi spezzettandoli in **segmenti** e aggiungendo loro un'**intestazione**; il prodotto è passato al livello di rete che li incapsulerà in un pacchetto detto **datagramma** e inviato a destinazione.

Lato server, invece, il livello di rete estrae i segmenti dai datagrammi ottenuti e li passa al livello di trasporto, il quale li elabora, estraendo i dati per renderli disponibili all'applicazione destinataria.

Come già menzionato in precedenza, il livello di trasporto fornisce due protocolli:

- **Transmission Control Protocol (TCP)**: Servizio orientato alla connessione, che garantisce la consegna ordinata di tutti i segmenti.
- **User Datagram Protocol (UDP)**: Servizio non orientato alla connessione, non affidabile.

Entrambi separano il messaggio originale in segmenti e aggiungono un'intestazione di forma diversa, la quale sarà trattata nelle seguenti sezioni. Prima di discuterne è tuttavia necessario chiarire in che modo i dati vengono trasferiti, ovvero come il servizio di trasporto a livello di rete possa diventarne un'altro da processo a processo per le applicazioni.

Ricordiamo che il livello di trasporto invia i dati agli host tramite un socket, il quale ha un identificatore univoco il cui formato dipende dal protocollo utilizzato. Ciascun segmento dei dati originali presenta vari campi; lato ricevente, il livello di trasporto li analizza per identificare il socket di ricezione e dirigervi il segmento. Definiamo questo

compito **demultiplexing**, mentre il suo duale, ovvero il raggruppare i segmenti ed incapsularli con un'intestazione per inviarli al livello di rete, è detto **multiplexing**. Con questi presupposti possiamo ora analizzare nel dettaglio i due protocolli di trasporto.

3.2 Protocollo TCP

Il **Transport Control Protocol** (TCP) è un protocollo orientato alla connessione perché richiede un preliminare scambio di messaggi per la sua instaurazione, ed è ritenuto affidabile in quanto si assicura la consegna ordinata di ogni dato tramite ricevute inviate dall'end-host ricevente.

Supponiamo ora che il processo di un host voglia comunicare con un altro; in primo luogo, l'applicativo del client informerà il suo livello di trasporto di voler stabilire una connessione verso un processo nel server e gli invierà quindi un segmento TCP. Il server risponderà con un altro segmento, alla cui ricezione da parte del client, ne invierà un terzo. Inviando tre segmenti, questa fase è detta **three-way handshake** e le sue specifiche saranno trattate nel dettaglio più avanti nella sezione.

Instaurata una connessione TCP, i due processi applicativi potranno scambiarsi i dati bidirezionalmente tramite il socket aperto, in base ad un valore specifico, detto **maximum segment size** (MSS), che indica la massima quantità di dati prelevabili e posizionabili in un singolo segmento a livello di applicazione. È un valore impostato nel three-way handshake determinando la lunghezza del frame più grande che può essere inviato a livello di collegamento, la **maximum transmission unit** (MTU).

TCP accoppierà poi ogni blocco di dati (**payload**) ricevuto dal livello superiore con un'intestazione **TCP**, formando i segmenti da inviare a livello di rete, ivi incapsulati a loro volta in **datagrammi IP**. Quando l'end-host ricevente riceve un segmento, i suoi dati sono memorizzati in un buffer di ricezione il quale è letto dal livello applicativo.

Analizziamo nel dettaglio la struttura dell'intestazione TCP; comunemente occupa $20B$ e presenta i seguenti campi:

- **Porta sorgente/destinazione:** Identificatori dei processi coinvolti nella comunicazione. Ciascuno di questi valori è rappresentato da $16b$, rendendo possibile usare 65536 porte diverse, le quali si dividono in due tipi:
 - **Statiche:** Identificativi utilizzabili esclusivamente lato server associati a protocolli definiti da uno standard come HTTP o SMTP. Comprendono i valori dell'intervallo $[0, 1023]$.
 - **Dinamiche:** Identificativi assegnati dal sistema operativo al lato client quando viene aperto un socket. Prendono un numero casuale dall'intervallo $[1024, 65535]$.

- **Numero di sequenza/acknowledgement:** Utilizzati per implementare l'affidabilità del protocollo, richiedono entrambi $32b$. Il numero di sequenza indica l'offset rispetto al primo byte del segmento e consente di capire come devono essere ricostruiti¹. L'offset è calcolato sommando un **initial sequence number**, un numero casuale generato dal sistema operativo che rimane costante, con il numero di sequenza del segmento. Il numero di acknowledgement indica invece il primo byte del segmento che la destinazione si aspetta di ricevere.
- **Offset intestazione:** Specifica la lunghezza dell'intestazione TCP in multipli di $32b$.
- **Flag:** Questo campo contiene dei segnalatori con logica di codifica 1-hot. **ACK** segnala che un segmento contiene un acknowledgement per un segmento ricevuto correttamente. **RST**, **SYN** e **FIN** sono invece usati per impostare e chiudere la connessione. L'insieme richiede $6b$.
- **Finestra di ricezione:** Usata per il controllo di flusso, indica il numero di byte che il destinatario è disposto ad accettare. Richiede $16b$.
- **Checksum Internet:** Richiesto per il controllo di errori nel segmento. Si chiama una funzione che prende l'intestazione come input; il suo output è assegnato a questo campo. Si tratta di un controllo attuabile da entrambi gli host e se l'output risultante è lo stesso, è altamente probabile che i dati siano corretti. Se differiscono, il segmento è scartato.
- **Opzioni:** Facoltativo e di lunghezza variabile, è utilizzato quando i due host devono determinare la lunghezza dell'MSS.

Definita l'intestazione, ora possiamo andare più nel dettaglio in merito all'instaurazione delle connessioni TCP. Il three-way handshake si compone di più elementi: in primo luogo, di un messaggio di **sincronizzazione** inviato dal client, contenente i parametri delle porte, numero di sequenza, numero di acknowledge e il flag SYN posto a 1. Segnala la volontà di aprire una connessione.

Il server risponderà con un messaggio di acknowledge sincronizzazione, dove sono contenuti i parametri delle porte invertiti, l' ISN_s ², acknowledge number e naturalmente i flag SYN e ACK posti a 1.

La stretta di mano si conclude con un messaggio di acknowledge da parte del client, il quale invierà i parametri delle porte, l' ISN_c , il flag ACK posto a 1 e come numero di acknowledge il valore $ISN_s + 1$.

¹Supponiamo di avere un messaggio di dimensione $5000B$ e che l'MSS sia di $900B$. Il primo segmento avrà 0 come numero di sequenza, il secondo 900, il terzo 1800 e così via.

²Indichiamo ISN_c e ISN_s come initial sequence numbers di client e server rispettivamente. In particolare, $ISN_s = ISN_c + 1$.

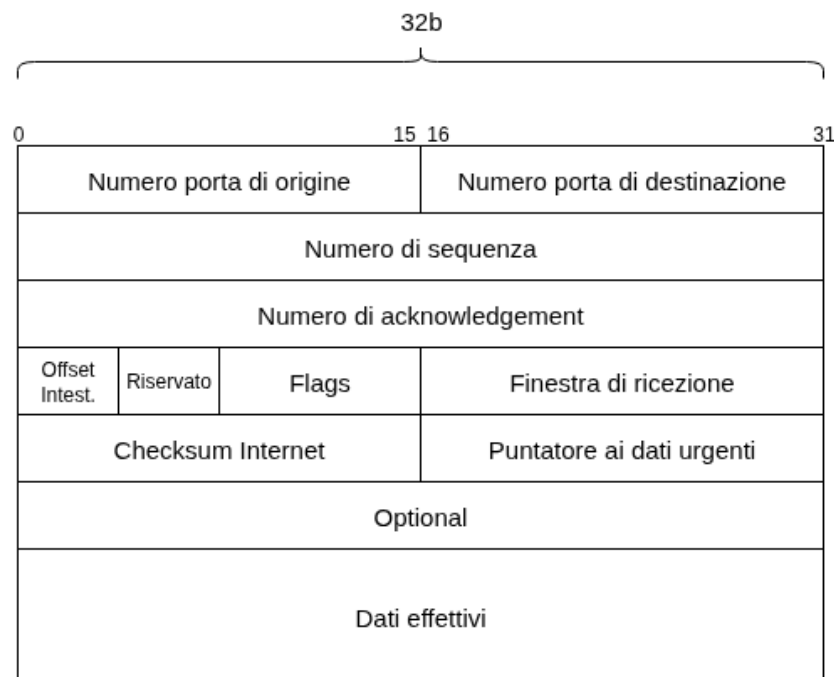


Figura 3.1: Intestazione dei segmenti TCP

Durante il three-way handshake è inoltre deciso il parametro di maximum segment size per evitare che si sforino le capacità delle schede di rete, ed è scelto il valore più piccolo delle due entità.

La richiesta di chiusura della connessione, come per il suo duale, richiede uno scambio di messaggi di header TCP, ed è una procedura che avviene indipendentemente nei due host. Si utilizzano messaggi di FIN e ACK con i relativi flag posti a 1 e possono avvenire i seguenti casi:

- Se il client è il primo a inviare un FIN, il server ritornerà un ACK e la connessione sarà chiusa lato client. L'altro host invierà poi a sua volta il FIN ed il client risponderà con un ACK. La connessione si può considerare chiusa solo dopo questo scambio.
- Il primo host invia un messaggio di FIN e il secondo ritornerà un ACK, chiudendo una direzione del socket. Quando la procedura è ripetuta in viceversa, la connessione TCP sarà del tutto chiusa.
- Una procedura più rapida vede il primo host inviare un FIN ed il secondo ritornare un FIN-ACK. Ricevuto quest'ultimo messaggio, il primo host risponderà con un ACK a sua volta, chiudendo la connessione.

Chiediamoci adesso quali siano le modalità procedurali che rendono il TCP un protocollo affidabile. Adotta in primo luogo la tecnica del **positive acknowledgement with retransmission**, dunque ad ogni segmento inviato deve corrispondere un responso positivo dall'altro host. Quando non è ricevuto, si ritenta l'invio fino a quando non lo si ottiene.

Il riscontro è un messaggio di acknowledgement che, come detto prima, contiene il relativo flag posto ad 1 ed il primo byte del segmento che la destinazione si aspetta di ricevere. Ciò significa che se lato client è stato inviato un sequence number 44, il numero di acknowledge inviato dal server sarà 45. Quando non è ricevuto questo valore, si attiva un timer chiamato **Retransmission Time-out** (RTO), al termine del quale si invia nuovamente il messaggio precedentemente non corrisposto.

Il valore di RTO è inizialmente posto a $5000ms$ preventivando eventuali problemi legati a fattori come distanza geografica o problemi di rete. Non è tuttavia un numero fisso; infatti, client e server misurano il Roundtrip time ad ogni segmento inviato e usano il primo come base di calcolo per aggiornare una variabile chiamata **smoothed-RTT** (SRTT) oppure Exponential Weighted Moving Average (EWMA) secondo la formula:

$$SRTT_{\text{attuale}} = \alpha SRTT_{\text{precedente}} + (1 - \alpha) RTT_{\text{istantaneo}} \quad \text{con } 0 < \alpha < 1$$

SRTT è una stima del valore medio di Roundtrip time e si utilizza per calcolare l'RTO con la formula:

$$RTO = \beta \cdot SRTT \quad \text{con } \beta \text{ generalmente uguale a } 2$$

Il quale, in caso di perdita di un segmento, è raddoppiato. Questo meccanismo è necessario perché potremmo trovarci nelle condizioni di inviare segmenti ad un'applicazione con bassa velocità di lettura del suo buffer di ricezione, ma allora dovremmo rallentare la velocità di trasmissione, facendo risultare la connessione più lenta. Come è possibile avere un'alta velocità di trasmissione senza mandare il buffer in overflow?

Le risposte offerte dal protocollo TCP vengono nella forma di **controllo di flusso** e **controllo di congestione**; il primo è un'azione preventiva per evitare la congestione e riduce la probabilità di perdere dei segmenti, mentre il secondo è una reazione in caso di congestione della rete, dunque quando non si ricevono riscontri per i segmenti inviati.

Il controllo di flusso è un meccanismo che consente anche l'invio di più segmenti alla volta grazie all'implementazione di una **finestra di ricezione scorrevole** (RCVWND) con un relativo parametro che salva la sua dimensione. Si tratta del numero massimo di segmenti inviabili al buffer del destinatario; se per esempio la dimensione corrisponde a tre segmenti, il mittente potrà inviare i segmenti s_1, s_2, s_3 in una volta sola, alla cui ricezione da parte del server, risponderà con tre numeri di acknowledge di fila. Ad ogni riscontro ricevuto, la finestra si sposterà di una posizione, per consentire l'invio dei

segmenti successivi, fino alla consegna dell'intero pacchetto. La formula della velocità di trasmissione diventa quindi:

$$v_t = RCVWND / RTT$$

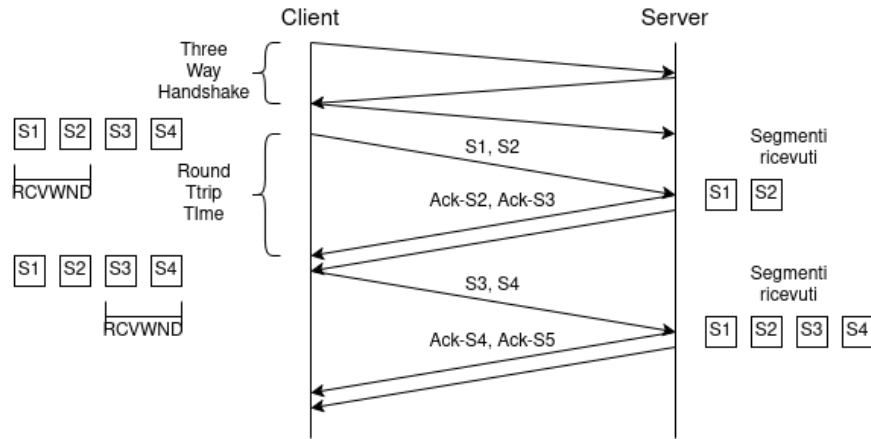


Figura 3.2: Esempio controllo di flusso

Non sempre si può ottenere il caso ideale appena discusso; è possibile avere **perdite** o **ritardi** di segmenti durante i tentativi della loro trasmissione. Per il primo scenario supponiamo che il client voglia inviare una sequenza di tre segmenti s_1, s_2, s_3 e che il terzo venga perso per qualche motivo.

In tal caso, il server ritornerà correttamente i numeri di acknowledge per i primi due segmenti senza inviare l'ultimo. L'RTO per il terzo segmento è quindi ancora attivo. Ricevuti i riscontri, la finestra di ricezione si sposterà di due posizioni e il client potrà inviare i successivi due elementi s_4, s_5 , correttamente ricevuti dal server, il quale continuerà a ritornare l'acknowledge number per il segmento mancante: *ack3*. Il mittente attenderà quindi lo scadere dell'RTO per ritrasmettere il segmento s_3 , ed una volta recepito dal server, ritornerà *ack6*, poiché ha già ricevuto tutti i segmenti fino a s_5 .

Generalmente, quando un server riceve pacchetti diversi da quelli comunicati tramite l'acknowledge number, continua a reinviare quest'ultimo fino a quando non è recepito. In caso di ritardi infatti, se un client invia tre segmenti s_1, s_2, s_3 come prima, ma il secondo ha un ritardo, il server invierà *ack2* fino a quando non gli sarà consegnato.

Ma in che modo è determinata la dimensione della finestra di trasmissione? Non è possibile avere un valore determinato, bisogna infatti adattarlo in base alla situazione corrente della rete. Ciò avviene grazie ai riscontri; la loro ricezione o assenza è il metro per l'adattamento.

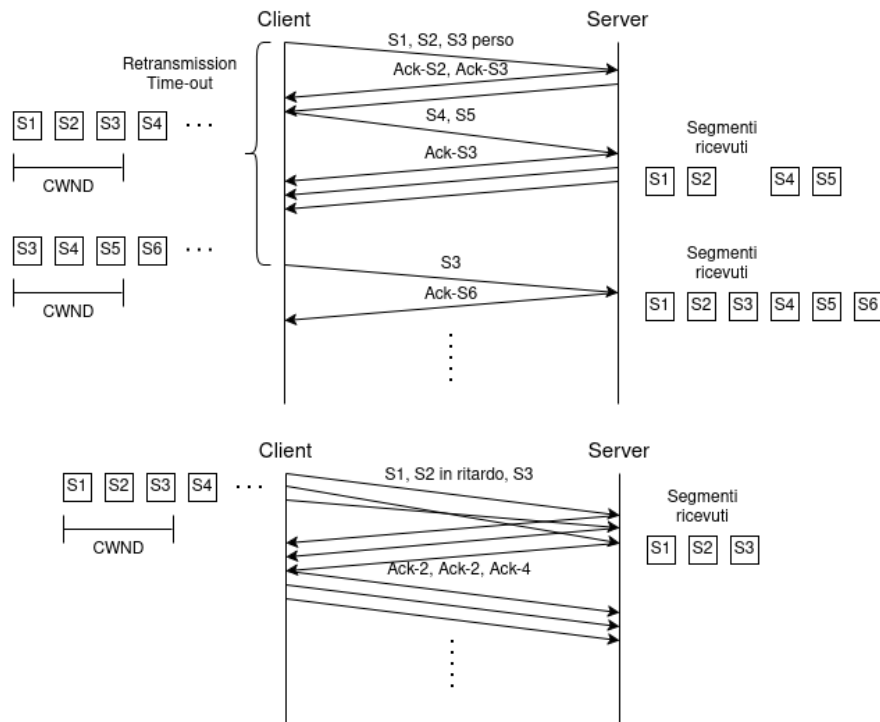


Figura 3.3: Perdita e ritardo di pacchetti

In genere, se i riscontri sono ricevuti regolarmente, è un segnale che la rete è in grado di gestire il traffico inviato e si potrebbe pensare di aumentare la dimensione della finestra. In caso contrario, quindi in presenza di perdite, alla scadenza dell'RTO ne si diminuisce la dimensione. Le regole per la gestione di questo meccanismo sono:

- **Slow-Start:** Oltre a spostare la finestra, aumenta la sua dimensione di un segmento per ogni riscontro correttamente ricevuto.
- **Congestion avoidance:** Ad ogni riscontro ricevuto, aumenta la dimensione della finestra di $1/w$. Per esempio, partendo con una dimensione $w = 2$, alla ricezione dei due riscontri si incrementa la grandezza di $2 \cdot 1/2 = 1$ segmento.
- **Vanilla TCP:** Quando non si è ricevuto un riscontro entro l'RTO, la dimensione è ridotta a 1 e si ricomincia la trasmissione a partire dal segmento perso.
- **Fast retransmit/Fast recovery:** Quando non si è ricevuto un riscontro entro l'RTO, si dimezza³ la dimensione attuale della finestra e si ricomincia la trasmissione a partire dal segmento perso.

³Generalmente si approssima il risultato per difetto, ma è contemplata, previa segnalazione, l'approssimazione per eccesso.

Viene usato un **algoritmo** specifico per il **controllo** della **congestione** e richiede i parametri discussi prima: congestion window (CWND), la dimensione della finestra; retransmission time-out (RTO); receiver window (RCVWND), la finestra massima di ricezione segmenti; slow-start threshold (SSTHRESH), soglia per determinare l'utilizzo di slow-start o congestion avoidance.

Algorithm 1 TCP-WindowManager()

```

cwnd  $\leftarrow$  1
rcvwnd  $\leftarrow$  valore comunicato ad ogni riscontro
ssthresh  $\leftarrow$  rcvwnd
rto  $\leftarrow$  valore misurato con  $\beta \cdot SRTT$ 

Client invia un numero cwnd di segmenti
if Riscontro ricevuto then
  if cwnd < ssthresh then                                 $\triangleright$  Uso lo slow-start
    cwnd  $\leftarrow$   $\min(\text{cwnd} + \text{ackNum}, \text{rcvwnd}, \text{ssthresh})$ 
  else                                                         $\triangleright$  Uso la congestion avoidance
    cwnd  $\leftarrow$   $\min(\text{cwnd} + \text{ackNum}/\text{cwnd}, \text{rcvwnd})$ 
  end if
else                                                         $\triangleright$  Se non arrivano riscontri e scade l'RTO
  ssthresh  $\leftarrow$  cwnd/2
  cwnd  $\leftarrow$  1                                               $\triangleright$  In caso di vanilla TCP
  rto  $\leftarrow$  2 * rto
end if
  
```

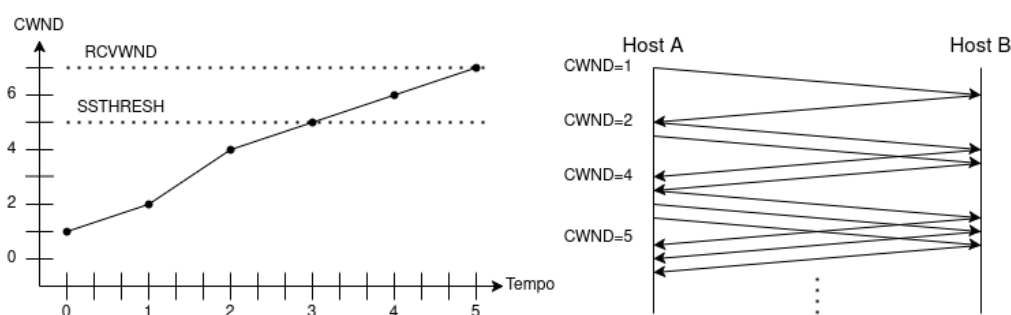


Figura 3.4: Panoramica algoritmo controllo congestione

3.3 Protocollo UDP

Il protocollo **User Datagram Protocol** (UDP) ha una struttura nettamente più semplice rispetto al TCP: diciamo anzitutto che è **connectionless** perché non effettua un preliminare scambio dei messaggi prima di inviare dati al server. UDP riceve i messaggi, aggiunge porta di origine e destinazione e passa il segmento a livello di rete, dove viene incapsulato e inviato in modalità "best-effort". Ciò significa che non è garantita la consegna dei segmenti, rendendolo un protocollo inaffidabile.

Tuttavia, non significa che non abbia comunque i suoi utilizzi. UDP è un protocollo utilizzato per le applicazioni **tolleranti alle perdite** grazie ad alcuni suoi pregi:

- Richiede un controllo più preciso a livello applicativo su quali dati sono inviati e quando.
- In quanto connectionless, non genera un ritardo per l'instaurazione della connessione.
- Richiede minor spazio per l'intestazione dei pacchetti.

Quindi, sebbene a primo impatto possa sembrare inutilizzabile, UDP è un protocollo importante per il funzionamento di molte applicazioni che richiedono tempi di risposta rapidi e in tempo reale. Per esempio, in quelle di streaming audio si riceve il segnale, che è quantizzato e discretizzato, per poi essere diviso in pacchetti ai quali è aggiunto l'header. Vengono inviate al secondo host, il quale potrà ricostruirlo ordinatamente, ma in caso di perdita, lascerà del silenzio. La velocità di trasmissione dei dati dipende quindi dalla loro velocità di generazione.

L'header, come già detto, di dimensione minore rispetto al TCP, presenta quattro campi: porta di origine, porta di destinazione e checksum, il cui funzionamento è identico al TCP, e quello di **lunghezza dei dati** per indicare la loro dimensione.

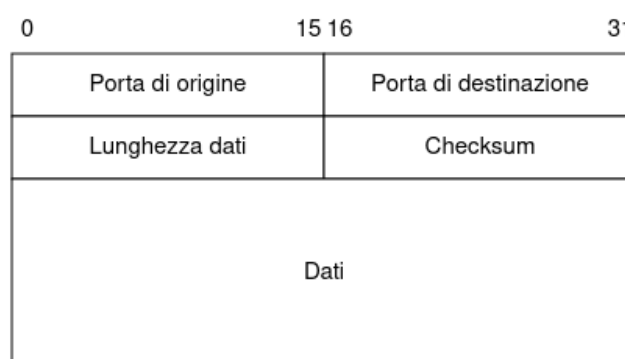


Figura 3.5: Panoramica intestazione UDP

3.4 Esercizi svolti

Esercizio 1: Sia un'applicazione A che desidera trasferire $96000B$ ad una seconda applicazione B. Si suppone che la connessione sia già instaurata e presenti i seguenti parametri:

- Maximum Segment Size: $1000B$.
- Receive Window: $32000B$, rimane costante per tutta la trasmissione.
- Slow Start Threshold: $RCVWND_i/2$.
- Round-trip time: $0,5s$, rimane costante per tutta la trasmissione.
- Retransmission time-out: $2RTT$, raddoppia in caso di perdite sequenziali.

Sono inoltre presenti dei down di rete durante gli intervalli di tempo $(3, 3.5)$, $(7, 7.5)$. Valutare l'evoluzione temporale della congestion window fino al termine della trasmissione.

Lo svolgimento di questo esercizio è dato interamente dall'algoritmo di slow start, per il quale valgono le seguenti regole generali:

- La dimensione della congestion window aumenta esponenzialmente finché il valore è minore dello slow start threshold.
- Arrivati ad un valore maggiore o uguale al threshold, si pone il valore di quest'ultimo alla finestra, per poi procedere aumentando la sua dimensione di 1 ad ogni RTT.
- Con eventuali down di rete, il threshold ottiene il valore dato dal rapporto tra dimensione della finestra corrente e 2, poi la dimensione della finestra torna a 1. Inoltre, si potrà ricominciare a trasmettere solo dopo un RTO.

Sapendo questo, iniziamo ottenendo il valore in segmenti dei dati da trasmettere, dividendo i Byte totali con la MSS:

$$\text{Segmenti: } 92000/1000 = 92, \quad \text{RCVWND: } 32000/1000 = 32, \quad \text{SSTHR: } 32/2 = 16$$

Se disegnato correttamente, l'evoluzione temporale è la seguente:

- $1 + 2 + 4 + 8 + 16 + 17 + 18(0) = 48$. I 18 segmenti sono trasmessi ma non sono stati recepiti a causa del down di rete. Non è stato quindi ricevuto alcun acknowledgement. Si attiva l'RTO, si pone $SSTHR_2 = 18/2 = 9$ e $CWND = 1$.

- $48 + 1 + 2 + 4 + 8 + 9 + 10 + 11(0) = 82$. Gli 11 segmenti hanno subito la stessa sorte sovramenzionata causa down di rete. Scaduto l'RTO, abbiamo che $SSTHR_3 = 11/2 = 4,5$ e $CWND = 1$. L'approssimazione del threshold è arbitraria, previa segnalazione si può approssimare per eccesso o difetto. In questo esempio, approssimiamo per difetto, quindi $SSTHR_3 = 4$.
- $82 + 1 + 2 + 4 + 5 + (6 - 4) = 96$. La dimensione finale della congestion window risulta come:

$$CWND_f = CWND_c + \text{ackRicevuti}/CWND_c = 6 + 2/6 = 6,33 = 6$$

L'ultimo acknowledge è poi ricevuto con il termine del round-trip time attivato con l'invio degli ultimi $(6 - 4) = 2$ segmenti. Il tempo di trasmissione totale è quindi $t = 10,5s$

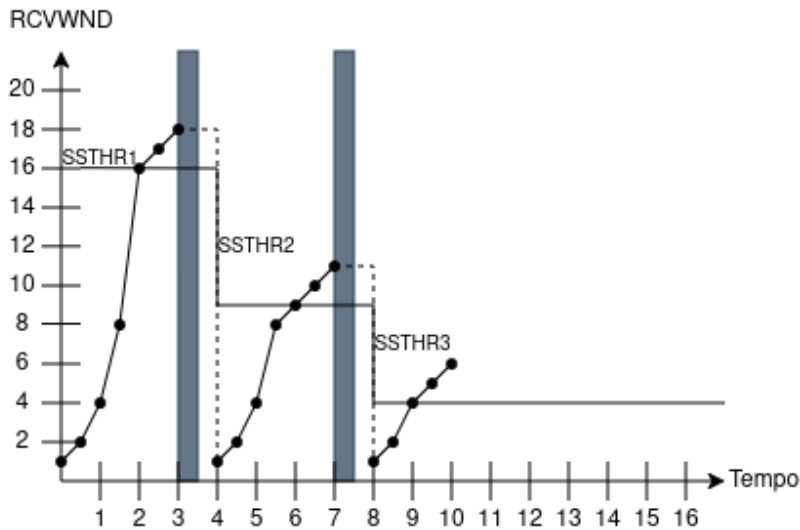


Figura 3.6: Evoluzione trasmissione esercizio 1

Esercizio 2: Sia un'applicazione A che desidera trasferire $46500B$ ad una seconda applicazione B. Si suppone che la connessione sia già instaurata e presenti i seguenti parametri:

- Maximum Segment Size: $1500B$.
- Receive Window: $24000B$, rimane costante per tutta la trasmissione.
- Slow Start Threshold: $RCVWND_i/2$.

- Round-trip time: $0,5s$, rimane costante per tutta la trasmissione.
- Retransmission time-out: $2RTT$, raddoppia in caso di perdite sequenziali.

Sono inoltre presenti dei down di rete durante gli intervalli di tempo $(1.5, 3.5)$, $(7, 7.5)$. Valutare l'evoluzione temporale della congestion window fino al termine della trasmissione.

Stesso identico procedimento di prima; trasformiamo i Byte in segmenti:

Segmenti: $46500/1500 = 31$, RCVWND: $24000/1500 = 16$, SSTHR: $16/2 = 8$

Se disegnato correttamente, l'evoluzione temporale è la seguente:

- $1 + 2 + 4 + 8(0) = 7$. Gli 8 segmenti sono stati persi a causa del down di rete. poniamo $SSTHR_2 = 8/2 = 4$ e $CWND = 1$; parte inoltre l'RTO, ma non è sufficiente per superare il problema. Si raddoppia, quindi, arrivando a $RTO = 2s$, dimezzando ulteriormente il threshold. Si supera il down.
- $7 + 1 + 2 + 3 + 4 + 5 + 6(0) = 22$. I 6 segmenti subiscono la medesima sorte. $SSTHR_4 = 6/2 = 3$, $CWND = 1$. Riparte di nuovo l'RTO di $1s$.
- $22 + 1 + 2 + 3 + (4 - 1) = 31$. All'ultima finestra rimanevano solo 3 segmenti da trasmettere.

Dimensione finale della $CWND_f = 4 + 3/4 = 4,75$, tempo di trasmissione totale $t = 10s$ al termine della ricezione dell'ultimo acknowledgement.

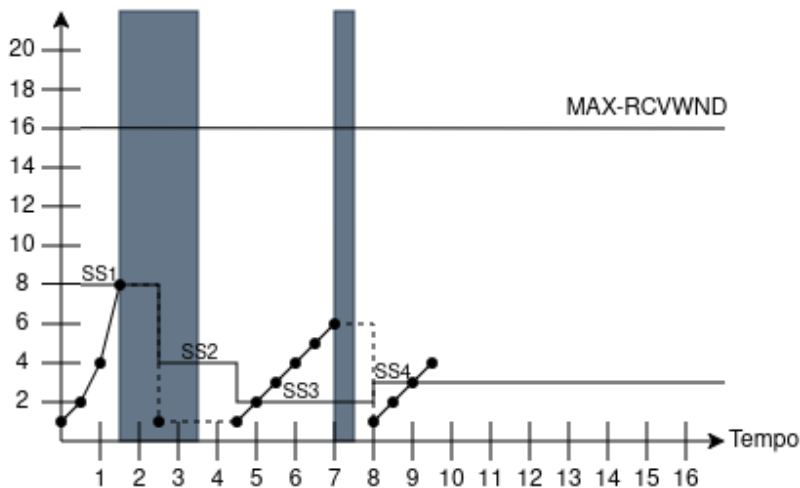


Figura 3.7: Evoluzione trasmissione esercizio 2

Esercizio 3: Sia un'applicazione A che desidera trasferire $104000B$ ad una seconda applicazione B. Si suppone che la connessione sia già instaurata e presenti i seguenti parametri:

- Maximum Segment Size: $1200B$.
- Receive Window: $24000B$, rimane costante per tutta la trasmissione.
- Slow Start Threshold: $RCVWND_i$.
- Round-trip time: $0,5s$, rimane costante per tutta la trasmissione.
- Retransmission time-out: $2RTT$, raddoppia in caso di perdite sequenziali.

Sono inoltre presenti dei down di rete durante gli intervalli di tempo $(3.5, 4)$, $(6.5, 10.5)$. Valutare l'evoluzione temporale della congestion window fino al termine della trasmissione. Non credo ci sia bisogno di ripetermi, no?

$$\text{Segmenti: } 104000/1200 = 87, \quad RCVWND: 24000/1200 = 20 = SSTHR$$

Trasmissione:

- $1 + 2 + 4 + 8 + 16 + 20 + 20 + 20(0) = 71$. Down di rete. You know the drill.
- $71 + 1 + 2 + 4 + 8 + 10(0) = 86$. Grosso down di rete ora. Tre volte scade l'RTT, diventando uguale a $4s$ e la SSTHR è ridotta a 2 segmenti.
- $86 + 1 = 87$.

Dimensione finale della CWND: $2 + 1/2 = 2.5$, tempo di trasmissione totale: $t = 14s$.

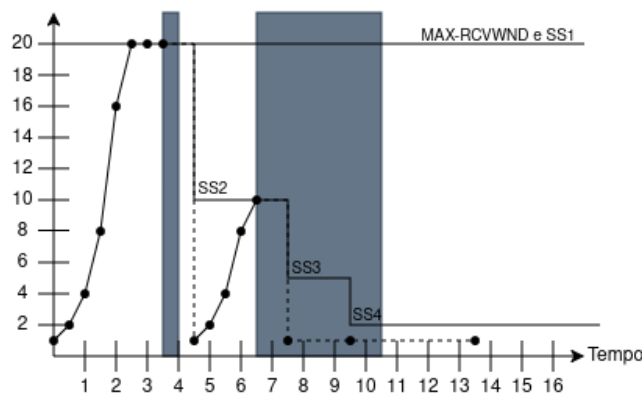


Figura 3.8: Evoluzione trasmissione esercizio 3

Capitolo 4

Livello network

4.1 Protocollo IP

L'**Internet Protocol** (IP) è l'unico protocollo disponibile per il livello di rete, il quale si occupa dell'incapsulamento dei segmenti ricevuti dal livello di trasporto, trasformandoli in **datagrammi**, e del loro trasporto da un host all'altro, router inclusi.

IP è un protocollo complesso che agisce su due piani: quello dei **dati** e quello di **controllo**, i quali interagiscono tra di loro. Utilizza inoltre algoritmi per l'instradamento dei datagrammi verso gli altri host, i quali saranno approfonditi nelle sezioni successive. L'intestazione usata per il protocollo IP si compone dei seguenti campi:

- **Versione:** Indica la versione utilizzata di IP; attualmente è in uso IPv4, ma i calcolatori sono pronti alla sua sostituzione con IPv6. Comprende $4b$.
- **Lunghezza intestazione:** I datagrammi possono contenere un numero variabile di opzioni, questo campo indica dove iniziano effettivamente i dati. Comprende $4b$, ma siccome è comune non avere opzioni in questi elementi, il tipico datagramma IP ha un'intestazione di $20B$.
- **Tipo di servizio:** Codice identificativo di una classe di servizio, consente di determinare il livello di priorità associato al datagramma per indirizzarlo verso il più appropriato buffer di uscita. Si comprende di $8b$.
- **Lunghezza del datagramma:** Indica la dimensione globale dell'intero datagramma e comprende $16b$.
- **Identificatore, flag e offset di frammentazione:** Campi richiesti per la frammentazione, una procedura necessaria per l'invio dei datagrammi anche in presenza di router con banda ridotta. Verrà trattata più nel dettaglio successivamente. Comprendono $16b$ totali.

- **Tempo di vita:** Contatore inizializzato a partire dall'host sorgente (tipicamente a valori come 64 o 128) che identifica il numero massimo di router che il datagramma può attraversare prima di essere scartato. Il valore è decrementato ad ogni hop eseguito e, raggiunto lo zero, è ritornato un messaggio di errore. Comprende 8b.
- **Protocollo:** Codice identificativo per determinare il protocollo usato nei dati trasportati nel payload. Prende 8b.
- **Header checksum:** Esegue la stessa funzione del checksum discusso durante il TCP. Richiede 16b.
- **Indirizzi IP sorgente e destinazione:** Utilizzati per determinare richiedente e ricevente dei datagrammi. Comprendono 32b l'uno.
- **Opzioni:** Utile per un'eventuale estensione dell'intestazione. Mette a disposizione altri 32b.

0	3	4	7	8	15	16	17	18	19	31
Versione	Lunghezza header		Tipo di servizio			Lunghezza totale				
Identificazione					Flags		Frammenti di offset			
Time To Live			Protocollo			Header checksum				
Indirizzo IP origine										
Indirizzo IP destinazione										
Optional										
Dati										

Figura 4.1: Panoramica intestazione IP

Come già avanzato nella descrizione dell'intestazione, è necessario parlare di **frammentazione** dei datagrammi, una procedura legata alla scheda di rete presente negli host, la quale definisce la MTU. Ha luogo quando la dimensione del pacchetto è maggiore

rispetto alla MTU dell'host e vede quest'ultimo frammentare il datagramma in più parti con dimensione compatibile a quella della sua scheda di rete.

Per attuare la procedura, si utilizzano i campi appositi dell'header IP; in particolare, **identificatore** mantiene un numero progressivo generato dalla sorgente e assegnato a ogni frammento, il **flag M** è utile per distinguere i frammenti secondo la logica:

- Se posto a 0, il datagramma non è stato frammentato oppure è l'ultimo frammento.
- Se posto a 1, il datagramma è uno dei frammenti.

Infine, **offset di frammentazione** indica la posizione del frammento rispetto al pacchetto originario, espresso in multipli di 8.

Supponiamo una MTU di $5000B$ ed un pacchetto di dimensione $4000B + 20B$ di header; naturalmente il datagramma non verrà frammentato e i tre campi discussi avranno i seguenti valori:

Identificatore: 476 Flag M: 0 Offset di frammentazione: 0

Durante il suo viaggio, il datagramma attraversa un router il cui link di uscita presenta una MTU di $1420B$; sarà necessario frammentare l'unità in tre parti, per le quali avremo:

- IP di sorgente e di destinazione uguali per ogni frammento.
- Flag M posto a 1 per i frammenti che non sono l'ultimo.
- Per il primo frammento un $\text{offset}_1 = 0/8 = 0$, per il secondo $\text{offset}_2 = 1400/8 = 175$ e per il terzo $\text{offset}_3 = 2800/8 = 350$.

Grazie alla frammentazione è possibile inviare i datagrammi in qualsiasi ordine e ricostruirli correttamente. Il riassemblaggio è compito dell'end-host di ricezione, il quale, alla ricezione di un frammento, farà partire un timer (generalmente di $250ms$), al termine del quale, se non sono stati ricevuti tutti i frammenti, scatterà tutto.

4.2 Algoritmi di routing

Come già chiarito, lo scopo del livello di rete è, data una destinazione, fare il possibile per consegnarle i datagrammi. Il livello di rete infatti non fornisce alcuna garanzia di consegna; esegue la sua mansione alla "best-effort" maniera, poiché è un compito assegnato al protocollo TCP. Esistono due tipi principali di consegna:

- **Diretta:** Effettuata tra due end-host all'interno di una stessa LAN.
- **Indiretta:** Effettuata tra due end-host appartenenti a LAN diverse, per la quale è necessario appellarsi al router di bordo.

Per poter determinare il tipo di consegna, un host è a conoscenza, oltre al proprio indirizzo IP, anche di quello del router di bordo e della maschera di rete. L'end-host effettua due operazioni di bit-wise *AND*: $IP_s \wedge \text{mask}$ e $IP_d \wedge \text{mask}$ per poi confrontare i risultati. Se sono uguali, la destinazione è all'interno della LAN e avverrà quindi una consegna diretta; alternativamente sarà indiretta.

Proprio in questo caso entra in gioco il **routing**: il processo di scoperta del cammino migliore da una sorgente a una delle possibili destinazioni. Per attuarlo, ogni router mantiene una **tabella di routing**, dove per ogni destinazione composta di tre campi:

- **Destinazione**: Il router al quale si vuole arrivare.
- **Hop successivo**: Il router attraverso il quale si deve passare per arrivare a destinazione.
- **Costo di attraversamento**: Autoesplicativo.

Il cammino migliore è determinato in base ai criteri adottati dall'ISP; alcuni scelgono di fare più hop e costare più tempo ma ridurre le risorse utilizzate, mentre altri potrebbero preferire un cammino con meno hop sacrificando più risorse. Non esiste la scelta migliore in assoluto.

Il calcolo del cammino minimo avviene prima dell'attraversamento ed è rappresentato mediante un'astrazione con i grafi. Andare nel dettaglio su questa forma o spiegare i relativi algoritmi esula dallo scopo di questa dispensa; per maggiori dettagli consultare quella di Algoritmi.

Data quindi una topologia composta da router (nodi) e collegamenti (archi), si deriva il grafo e l'ISP assegna i pesi in base al criterio adottato; in particolare se si sceglie il **numero di hop attraversati** è necessario porre tutti gli archi con peso uguale a 1 e il cammino minimo sarà il percorso con il valore minore. Alternativamente, scegliendo la **velocità di trasmissione** nominale del link, i pesi avranno un valore inversamente proporzionale alla loro banda, poiché maggiore è la velocità, minore sarà il tempo usato per il trasferimento dei datagrammi.

Esistono più algoritmi applicabili per la risoluzione del problema di calcolo del cammino minimo; in particolare:

- **Link-state routing algorithm**: Algoritmo "centralizzato", grazie al quale ogni router è a conoscenza della topologia della rete e calcola in autonomia il cammino minimo, scegliendo il link di uscita più appropriato.

Si implementa tramite l'algoritmo di Dijkstra, una procedura avida che sceglie il nodo non ancora visitato con distanza minima dalla sorgente e salva il percorso nella rispettiva tabella di routing. Impostate tutte le tabelle, i datagrammi saranno capaci di scegliere il link di uscita opportuno.

In questo contesto, i router si scambiano messaggi periodicamente per verificare se sono ancora raggiungibili; in caso di guasti o cambio di topologia, la relativa informazione è immediatamente propagata agli altri nodi, che aggiorneranno le proprie tabelle di routing. Il protocollo standard che i router devono rispettare per questo tipo di algoritmo è quello di **Open Shortest Path First (OSPF)**.

- **Distance vector routing algorithm:** Algoritmo distribuito dove i router sono a conoscenza esclusivamente delle altre entità adiacenti con le quali si scambiano messaggi per aggiornare le tabelle di routing e determinare il cammino di costo minimo, il cui peso è dato dalla somma dei costi di tutti i link attraversati nel percorso, ricalcolato ad ogni iterazione dell'algoritmo usato: Bellman-Ford.

Supponiamo il grafo pesato rappresentato nell'immagine successiva; in primo luogo si inizierà la tabella del nodo sorgente A , ottenendo i nodi adiacenti, hop successivi e relativo costo. Nella prima iterazione dell'algoritmo, si riceveranno le tabelle dei router adiacenti $B = [A7, B0, C1, E5]$ e $E = [A1, B5, D2, E0]$; per il calcolo del cammino minimo, la sorgente ottiene:

$$D(A, B) = \begin{cases} c(A, B) = 7 + D(B, B) = 0 \implies = 7 \\ c(A, E) = 1 + D(E, B) = 5 \implies = 6 \end{cases}$$

Visto che è stato trovato un costo minore passando da E , il nodo A aggiorna la tabella per la destinazione B con costo 6 segnando E come hop successivo. L'aggiornamento avviene per ogni destinazione contenuta nei distance vector (tabelle) ricevuti.

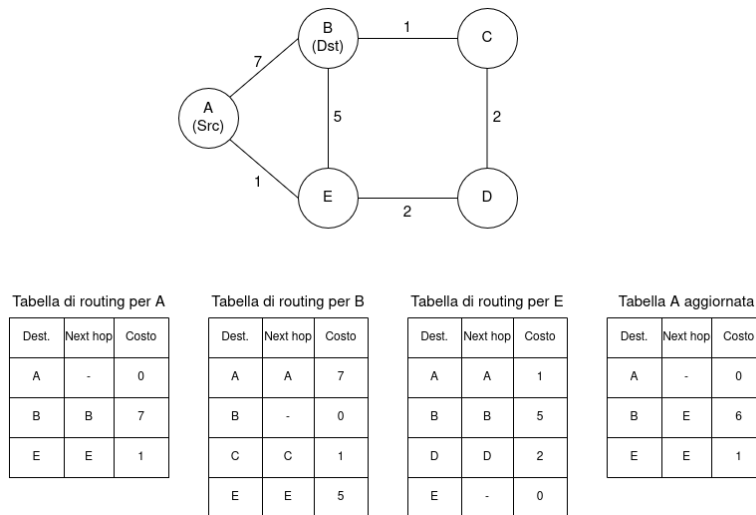


Figura 4.2: Algoritmo Distance Vector

- **Software-defined network:** Algoritmo effettivamente centralizzato, dove un'entità esterna, cosciente della topologia della rete, comunica con il router di sorgente fornendo tabelle di routing per la scelta del cammino minimo.

I messaggi periodici inviati tra i vari router per garantire il corretto funzionamento degli algoritmi seguono specifici protocolli il cui tipo è incapsulato all'interno dell'header IP. Tuttavia, questi algoritmi operano solamente all'interno del proprio insieme di reti e per questa ragione sono detti **intra-AS**; per le consegne indirette, **inter-AS**, è necessario appellarsi ai router di bordo, detti **gateway**, i quali instraderanno correttamente i datagrammi verso l'end-host desiderato della loro rete.

Per questi scopi è stato definito il protocollo standard condiviso da tutti i router: **Border Gateway Protocol (BGP)**. Supponiamo un router appartenente ad un AS, la sua tabella di routing conterrà i blocchi di indirizzi configurati da un protocollo intra-AS per le destinazioni facenti parte della propria rete, ma saranno presenti anche altre righe dedicate alle destinazioni extra-AS configurate dal protocollo BGP.

La configurazione è simile; l'hop successivo sarà sempre quello adiacente per poter proseguire nel cammino e bisognerà calcolare il costo minimo del percorso per arrivare al router di bordo. Non è infatti compito della rete di sorgente calcolare il cammino minimo anche per le altre, poiché queste ultime potrebbero avere policy differenti, impostate dagli amministratori di rete.

Il cammino minimo inter-AS risulta essere quindi una composizione di cammini minimi intra-AS, calcolati mediante questa procedura chiamata anche **Hot potato routing**; è possibile infatti dover attraversare più sistemi perché non tutte le reti hanno collegamenti diretti.

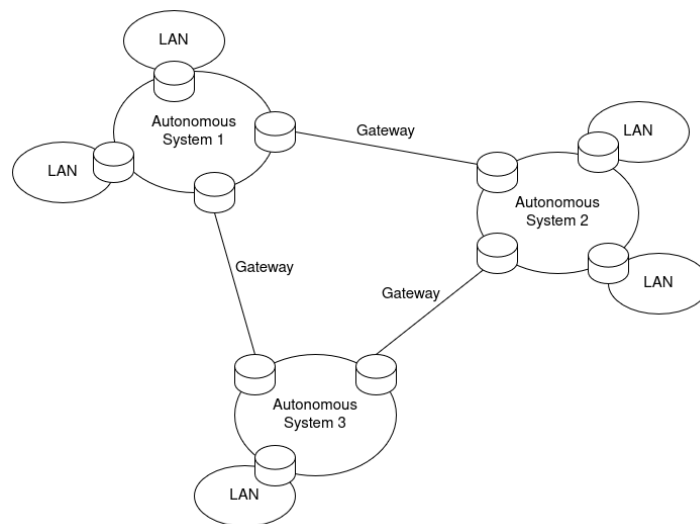


Figura 4.3: Panoramica inter-AS

4.3 Protocolli DHCP e ICMP

Il protocollo **Dynamic Host Configuration Protocol** è usato per la configurazione dell'host ogni volta che questo si connette a una rete. Tratta l'assegnamento dell'indirizzo IP e parametri per consentire il lavoro sulla rete scelta. Non entriamo in dettagli eccessivi riguardo la struttura del protocollo, tuttavia è necessario nominare due campi importanti della sua intestazione:

- **Message type**: Codice identificativo dei messaggi DHCP.
- **Transaction ID**: Codice identificativo dello scambio di messaggi DHCP generato casualmente dal richiedente.

L'entità per l'assegnazione degli indirizzi IP è detta **server DHCP**. La connessione a una rete avviene tramite lo scambio di quattro tipi di messaggio DHCP:

1. **DHCP Discover**: Il client invia un broadcast sulla porta UDP 67 a tutti gli host della rete, il quale sarà riconosciuto e inoltrato dal client DHCP al suo livello di collegamento dati. Il campo transaction ID è fondamentale per distinguere i richiedenti.
2. **DHCP Offer**: Un server DHCP riceve un messaggio di identificazione e risponde a sua volta con un broadcast contenente l'indirizzo offerto all'host che vuole instaurare una connessione, con il medesimo transaction ID ed un **lease time**¹ che definisce per quanto tempo può mantenere questo IP.
3. **DHCP Request**: Il client appena collegato sceglierà uno degli indirizzi offerti dai server DHCP e risponderà con un messaggio contenente l'IP del server, quello offerto e un nuovo transaction ID.
4. **DHCP Acknowledge**: La risposta finale del server che conferma i parametri richiesti.

Oltre all'indirizzo IP, i server DHCP forniscono anche la maschera della rete, il valore dell'IP del router di default e l'IP del server DNS.

Un altro protocollo, **Internet Control Message Protocol**, ha invece lo scopo di inviare messaggi che trasportano informazioni o errori a livello di rete. Il destinatario di questi messaggi può riconoscerli grazie alla segnalazione nel campo protocol presente nell'header IP.

Nella pratica è utilizzato come un logger, per esempio per segnalare la scadenza del TTL di un datagramma, prendendo l'IP sorgente del pacchetto e inviando un messaggio ICMP di tipo "TTL exceeded". Ha uso anche nel traceroute per distinguere i vari hop su cui misurare l'RTT.

¹Generalmente, parliamo di un'ora per le reti aziendali e 24h per quelle domestiche. Può essere rinnovato con DHCP request.

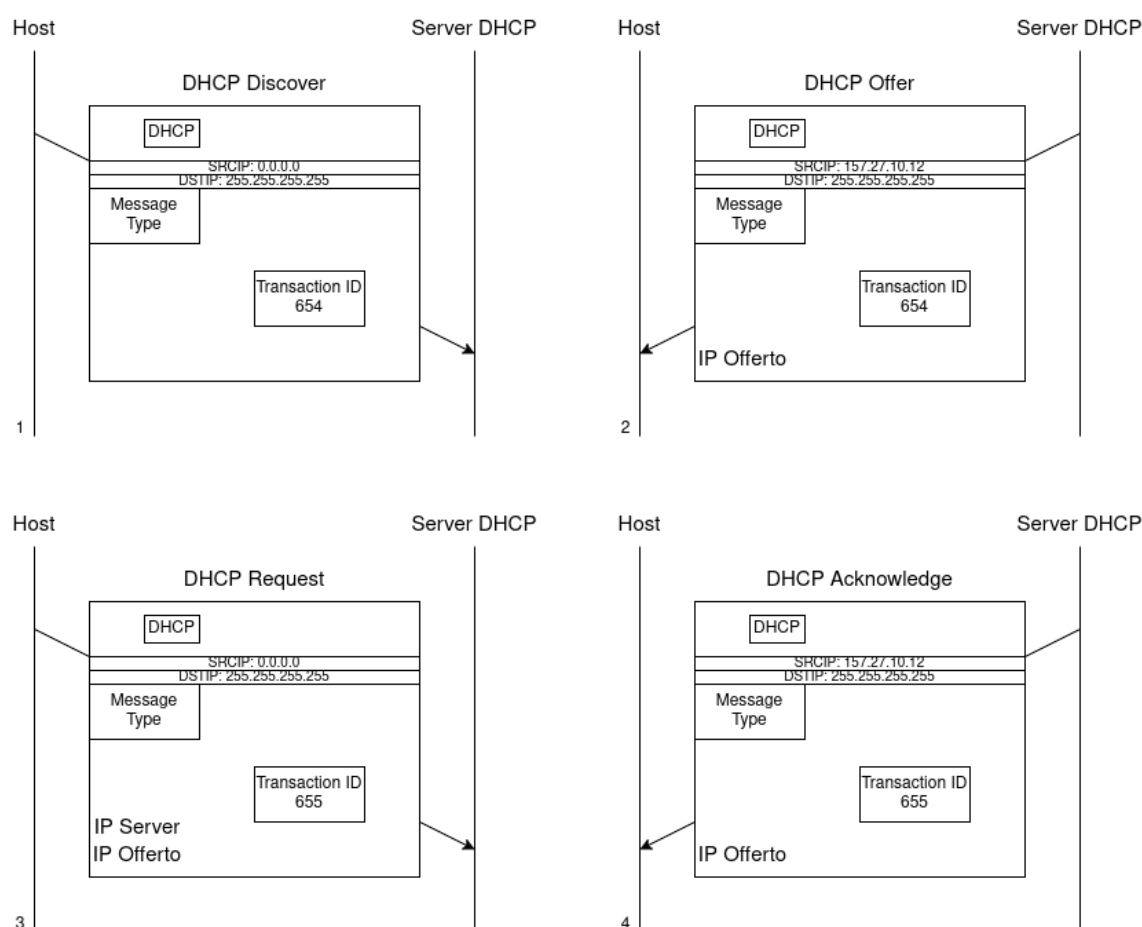


Figura 4.4: Scambio di messaggi DHCP

4.4 Network Address Translation e IPv6

Ad oggi si è potuto notare come l'insieme di indirizzi IP forniti dal protocollo IPv4, un totale di 2^{32} valori diversi, non siano sufficienti per garantire accesso a Internet per chi può; ragion per cui sono state proposte due soluzioni: l'introduzione di una classe di **IP privati**, la quale è attualmente in uso, e il cambio di protocollo in **IPv6** per garantire un totale di 2^{128} indirizzi. Non è ancora attiva, ma tutti gli elaboratori sarebbero capaci di usufruirne.

Approfondiamo la prima soluzione: definire una classe di indirizzi privati implica l'esistenza di relativi blocchi, i quali possono essere riutilizzati su più LAN. Le maschere sono in genere:

10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16²

La loro caratteristica di essere privati è dovuta al fatto che sono utilizzati in reti LAN private, distinte dalle altre grazie al proprio router di bordo con funzionalità di **Network Address Translation** (NAT). Il vantaggio principale è il risparmio sugli indirizzi pubblici, poiché riutilizzando quei blocchi non è necessario fornire un IP pubblico per ogni end-host. Inoltre, è resa possibile la comunicazione tra gli utenti della stessa rete senza l'appello di Internet.

Tuttavia, una rete non connessa a internet non ha alcuno scopo, ragion per cui il router NAT prende il ruolo di interfaccia con l'esterno. Si compone di un IP privato come gli altri host della rete per poter comunicare con loro, e di un IP pubblico rappresentante l'intera LAN privata per la comunicazione su Internet. Questo meccanismo è vantaggioso perché l'interfaccia dà l'impressione agli host di reti diverse che tutto il traffico sia generato esclusivamente dal router, celando loro la rete privata e, lato utente, non è cambiato nulla nelle modalità di interazione.

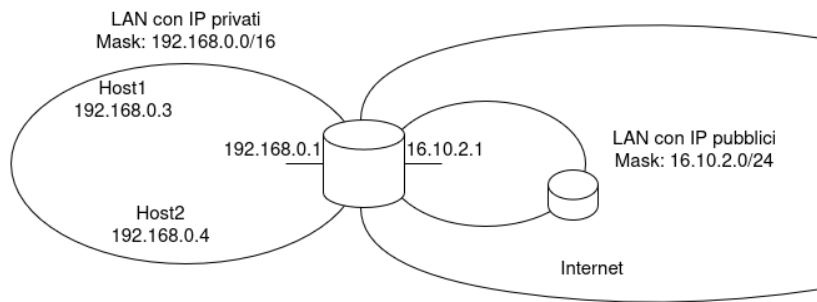


Figura 4.5: Panoramica NAT

Supponiamo che un end-host di una rete LAN privata generi un pacchetto verso una destinazione esterna. Questo avrà naturalmente un payload incapsulato dall'intestazione IP, dove nel campo IP sorgente è indicato quello privato dell'host, mentre come IP destinazione quello del ricevente pubblico.

Il pacchetto passerà necessariamente attraverso il router di bordo NAT, il quale sovrascriverà l'IP privato con la sua interfaccia pubblica, memorizzando il primo anche in un'apposita tabella, detta **Tabella NAT**. Inviato il pacchetto, il server risponderà con i campi IP dell'header invertiti e l'elemento sarà ricevuto dal router NAT, il quale ripristinerà l'IP destinazione con quello privato dell'host sorgente per farlo recapitare correttamente.

Queste informazioni tuttavia non sono sufficienti a coprire ogni istanza problematica del caso. Se infatti più host inviano un pacchetto alla stessa destinazione contempora-

²Riservato a link-local, diverso dai precedenti.

neamente, il router non sarebbe in grado di distinguere la sorgente corretta. Per questo motivo, la tabella NAT presenta tre campi aggiuntivi:

- **Porta sorgente:** Valore generato casualmente dall'host, è il principale discriminante³ per la distinzione delle richieste.
- **Porta destinazione:** Valore che dipende dal processo destinatario. E.g: 80 per messaggi HTTP.
- **Porta router:** Valore generato casualmente dal router NAT per sovrascrivere il campo di porta sorgente nell'header TCP.

Indirizzo IP privato sorgente	Indirizzo IP pubblico destinazione	Porta sorgente	Porta destinazione	Porta sorgente router
192.168.0.3	80.70.60.2	7312	80	20001
192.168.0.4	80.70.60.2	10412	80	20002
...	...	...	...	...

Figura 4.6: Tabella NAT

Questo schema di traduzione è detto **Network Address and Port Translation** (NAPT) e per evitare problemi di starvation, vede associato un timer ad ogni riga; se al suo scadere il router verifica che non ci sono messaggi relativi alla riga, questa viene cancellata. L'unico svantaggio di questo sistema è che, in quanto nascosti all'esterno, non è possibile contattare direttamente end-host delle reti private.

Supponiamo dunque di avere due reti LAN: L_1 privata e L_2 pubblica, i cui host sono intenzionati a scambiarsi messaggi. La comunicazione dalla prima alla seconda rete avviene nelle modalità discusse prima, ma viceversa si richiede il supporto di due server forniti dall'applicazione utilizzata:

- **Server Relay:** Assegna a ogni utente un identificativo unico stabile, indipendente dalla rete a cui si appartiene.
- **Server Directory:** Mantiene informazioni sullo stato degli utenti, in particolare il loro indirizzo IP e stato online/offline.

Quindi ogni volta che un utente vuole contattarne un altro connesso a una rete privata, richiede in primo luogo l'IP del destinatario al server Directory. Se questo è pubblico,

³Si tratta di un numero compreso in un intervallo specifico. Ciò comporta una remota ma esistente possibilità che due host scelgano la stessa porta, confondendo il router.

si ha una comunicazione normale con l'apertura di un socket; alternativamente ci si appoggia al server Relay per poter comunicare.

La soluzione del cambio di protocollo in IPv6 presenta la necessità di un cambio di intestazione, naturalmente. L'header di base ha una dimensione di $40B$ e contiene i seguenti campi:

- **Versione:** Consente di distinguere IPv6 da IPv4 e fornisce il significato dei campi successivi. Richiede $4b$.
- **Traffic class:** Gestisce le proprietà dei pacchetti in base al tipo di servizio dato dalla priorità. Richiede $8b$.
- **Flow label:** Numero assegnato al socket. È univoco e condiviso dagli IP di sorgente e destinazione, come anche le rispettive porte. Richiede $20b$.
- **Lunghezza payload:** Autoesplicativo. Richiede $16b$.
- **Next header:** Codice dal duplice significato. In mancanza di estensioni all'intestazione, identifica il protocollo trasportato nel payload; alternativamente, rappresenta il tipo di extension header. Richiede $8b$.
- **Hop limit:** Indica il numero di hop attraversabili dal frame prima di essere scartato. Corrisponde al campo TTL di IPv4 e richiede $8b$.
- **IP Sorgente:** Autoesplicativo. Ha $128b$ disponibili.
- **IP Destinazione:** Autoesplicativo. Ha $128b$ disponibili.

Il protocollo IPv6 dovrà eventualmente essere implementato quando la richiesta di indirizzi IP a $32b$ sarà diventata insostenibile. Una disponibilità di 2^{128} indirizzi sarà sufficiente per coprire le necessità dell'intera superficie terrestre, acqua inclusa.

Nel campo next header abbiamo poi nominato gli **extension header**: sono delle ulteriori intestazioni con un formato specifico. A noi interessano due campi in particolare:

- **Next header:** Campo dalla medesima funzione di quello in IPv6; è quindi capace di identificare ulteriori estensioni, potenzialmente, ma non realisticamente, ad infinitum. Richiede $8b$.
- **Next header length:** Indica la lunghezza dell'estensione. Richiede $8b$.

Le estensioni si possono utilizzare in vari contesti; per esempio, con la frammentazione, il campo next header di IPv6 indicherà il tipo di extension header contenente una parte dei dati frammentati. Un altro caso è con l'instradamento, dove sono indicati gli indirizzi IP dei router da attraversare.

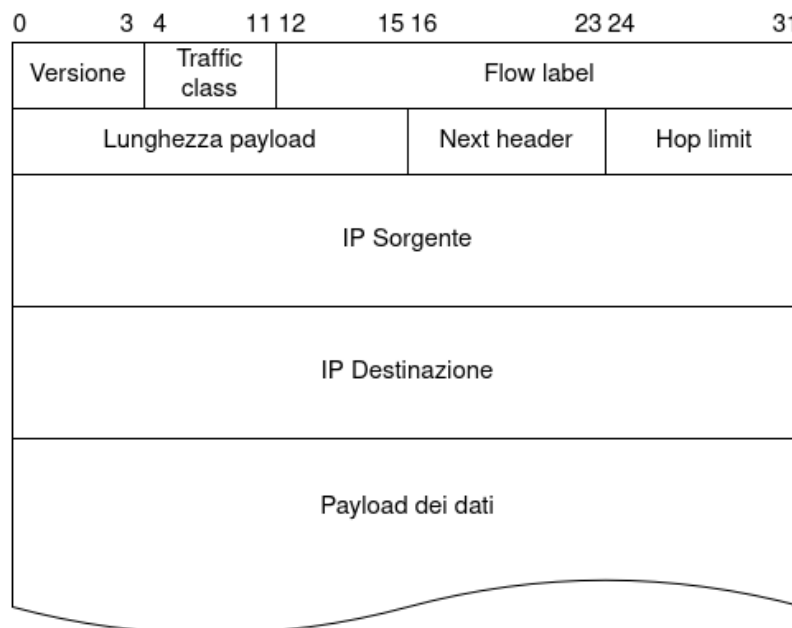


Figura 4.7: Intestazione IPv6

I vantaggi di questo costrutto riguardano in primo luogo la possibilità di estendere gli header con nuove funzionalità senza modificare quello di base, facendo scartare il frame a quei router non in grado di gestirlo. Ciò consente anche un aggiornamento graduale degli hardware. Inoltre, le estensioni sono utilizzate esclusivamente se necessarie.

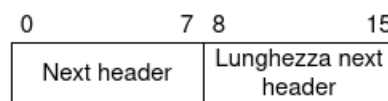


Figura 4.8: Estensione dell'intestazione

4.5 Esercizi svolti

Esercizio 1: Sia la seguente architettura di rete mostrata in immagine. Scrivere la tabella di routing per il router A considerando come metrica il numero di hop e assumendo che il router X abbia annunciato di poter raggiungere tutti gli host su Internet con 5 hop. Le LAN dispongono di un mezzo condiviso per il salto da una rete all'altra.

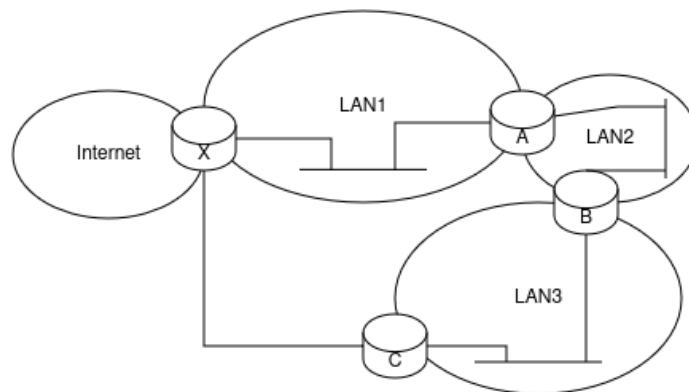


Figura 4.9: Architettura esercizio 1

Questo esercizio è molto più basato sul guardare l'architettura di rete piuttosto che di ragionamento. È stato richiesto di prendere come riferimento A; quindi basta vedere il numero di reti da esso attraversate per poter fornire costi verso una data destinazione.

Destinazione	Costo	Hop successivo
LAN1	1	diretta
LAN2	1	diretta
LAN3	2	B
Internet	6	X

Tabella 4.1: Tabella di routing per il router A

Capitolo 5

Livello di collegamento dati

5.1 Servizi e accesso al canale condiviso

Lo scopo del livello di **collegamento dati** è la gestione e risoluzione dei problemi associati alla trasmissione sul singolo hop, senza conoscere l'intera topologia della rete. Offre quindi servizi di:

- **Framing:** Molti protocolli di questo livello incapsulano i datagrammi all'interno di un frame prima di trasmetterlo, la cui struttura è definita dal primo.
- **Accesso al mezzo condiviso:** È necessario che i protocolli forniscano un **Medium Access Control** (MAC) per l'accesso ai collegamenti, il quale specifica le regole con cui immettere i frame nel socket. Saranno approfonditi nella sezione successiva.
- **Consegna affidabile:** I protocolli che forniscono questo servizio sono utilizzati in un contesto dove è comune trovare errori nella trasmissione, ed è quindi necessario implementare una funzione per la relativa gestione.
- **Rilevazione e gestione errori:** Può capitare che a causa di disturbi del segnale i dati vengano corrotti. Per questa ragione, molti protocolli forniscono un meccanismo per rilevare questi problemi grazie ad un bit di flag all'interno del frame, il quale sarà controllato dall'host ricevente.

I protocolli usati a livello di collegamento dati dipendono dalla tecnologia della porta di rete, come la rete **Wi-Fi** (IEEE 802.11), una rete fissa **Ethernet** (IEEE 802.3), oppure **Fibra ottica**, che fa uso di Synchronous Digital Hierarchy (SDH) o di Point to Point Protocol (PPP). È chiaro come questo livello sia strettamente legato all'hardware; infatti il protocollo è realizzato da un **adattatore di rete** (scheda di rete), nel quale un chip dedicato ne fornisce i relativi servizi, mentre lato software sono implementate le funzioni per comunicare con i livelli superiori.

Ogni host presenta più interfacce di rete per l'invio e la ricezione dei dati in base alle tecnologie, le quali corrispondono a un collegamento fisico specifico, definito da un **Media Access Control Address** (Indirizzo MAC), il quale sarà utilizzato negli header dei protocolli per la scelta del canale da utilizzare. In quanto condiviso, tuttavia, si presenta il problema di regolamentazione accessi e delimitazione delle trasmissioni. Supponiamo infatti un router di una rete domestica, al cui access point sono collegati più host, come PC, telefoni, console, etc; questi potrebbero entrare in contesa se utilizzano i servizi concorrentemente. La soluzione al problema deve cercare al meglio delle sue possibilità di:

- Minimizzare le collisioni fra le trasmissioni e quindi la perdita di informazioni.
- Massimizzare l'utilizzo del mezzo ed evitare sprechi di risorse.

La prima strategia proposta è quella di **allocazione statica**, la quale vede le risorse suddivise a priori tra i partecipanti. Degli esempi di questa implementazione sono il Time Division Multiplex (TDM), che divide il tempo in vari slot, ognuno dei quali associato ad ogni utente, e il Frequency Division Multiplex, che fa lo stesso, ma per le frequenze. La strategia statica non risponde appropriatamente alle nostre richieste, perché non massimizza l'utilizzo del mezzo; dividendo aprioristicamente le risorse, è inevitabile che quando non ci sia richiesta dagli host, lo slot ad essi associato sia inutilizzato. Inoltre, con aggiunta o rimozione di altri host sarebbe necessario ricalcolare la divisione delle risorse. La strategia attualmente in uso è quella di **allocazione dinamica**, che si divide a sua volta in due tecniche:

- **A turno:** Organizzano le stazioni (host) in un determinato ordine e si definisce un token che dona il diritto di trasmissione per un determinato intervallo di tempo. Scaduta la disponibilità, si passa all'utente successivo. Questa implementazione in particolare è chiamata **token-ring** e richiede di stabilire un ordine e intervallo di disponibilità del token. Inoltre, più stazioni sono presenti, meno tempo potrà essere dedicato alla trasmissione della singola.
- **A contesa:** Non implementano un coordinamento centralizzato e le varie stazioni si contengono l'utilizzo del canale di comunicazione. Si tratta della strategia attualmente in uso, della quale studieremo nella prossima sezione i protocolli Aloha, Slotted-Aloha e CSMA.

5.2 Algoritmi di accesso al canale condiviso

Il primo algoritmo discusso, come anche il più semplice tra tutti, è **Aloha**. Si tratta di uno schema distribuito che vede più stazioni indipendenti eseguire la stessa seguente procedura per la trasmissione delle trame. La verifica della collisione avviene tramite

”ascolto” del canale. Viene rilevato un valore di potenza, il quale, se è più alto del normale, indica che stanno già venendo trasmessi dei dati e bisogna attendere il termine dell’intervallo di tempo calcolato casualmente. Tuttavia, è necessario che le stazioni siano desincronizzate, perché se così non fosse, continuerebbero a trasmettere le loro trame senza successo. Aloha trova utilizzo ancora oggi grazie alla sua semplicità, per esempio per le chiamate telefoniche.

In uno scenario ideale, Aloha è eseguito in tempo lineare $\Omega(n)$, ma nella realtà è possibile utilizzare al massimo il 19% del suo potenziale. Definiamo inoltre **periodo di vulnerabilità** l’intervallo di tempo in cui una trama può subire collisioni, il quale dipende dall’algoritmo usato. Per quanto riguarda Aloha, ha durata uguale a due volte il tempo di trasmissione usato per la trama. In particolare:

- **Tempo di trama:** $T = \text{bitDaTrasmettere} / \text{velocitàTrasmissione}$
- **Periodo di vulnerabilità:** $V = 2T$

Algorithm 2 Aloha()

```

while Stazione ha dati da trasmettere do
  Trasmetti i dati
  if Canale è occupato then                                     ▷ Collisione verificata
    Estrai un tempo casuale da aspettare
  end if
  while Tempo non scaduto do
    Decrementa il tempo
  end while
end while
  
```

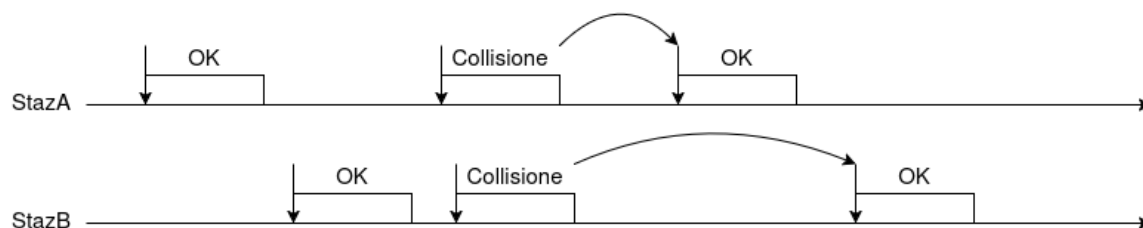


Figura 5.1: Evoluzione di Aloha nel tempo

Slotted-Aloha è una versione modificata del precedente algoritmo; suddivide il tempo in vari intervalli di dimensione uguale al tempo di trama T mantenendo la stessa dinamica di trasmissione senza ascolto del canale, ma permettendola solamente all'inizio di uno di questi slot.

In questo algoritmo una stazione rischia di subire collisioni solo se un'altra sta trasmettendo nello stesso slot, dunque il periodo di vulnerabilità corrisponde alla grandezza dello slot: il tempo di trama T . Ciò si traduce in un'efficienza migliore rispetto all'Aloha base, ovvero del 38%.

Algorithm 3 slotted-aloha()

```

while Stazione ha dati da trasmettere do
  Attendi l'inizio dello slot successivo e trasmetti i dati
  if Canale è occupato then
    Estrai un tempo casuale da aspettare
  end if
  while Tempo non scaduto do
    Decrementa il tempo
  end while
end while

```

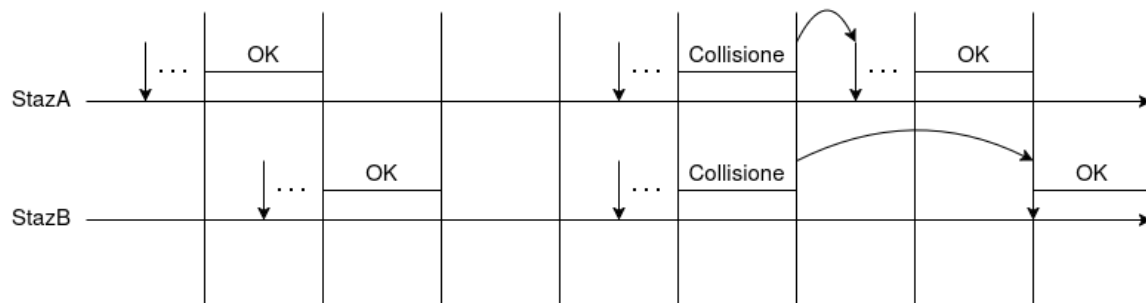


Figura 5.2: Evoluzione di Slotted-Aloha nel tempo

È possibile aumentare ulteriormente l'efficienza riducendo il numero di collisioni con l'algoritmo **Carrier Sense Multiple Access (CSMA) persistente**, la prima procedura che continua ad ascoltare il canale fin quando non è libero prima di trasmettere.

Sebbene il controllo sulle collisioni possa a primo impatto sembrare superfluo, queste possono avvenire quando due stazioni desiderano trasmettere una trama e attendono la terminazione di una precedente. Queste inizierebbero la trasmissione allo stesso tempo, collidendo. Per quanto una gestione migliore del canale rispetto ad Aloha, è per questa ragione, che con un grande numero di stazioni, l'utilizzo del mezzo risulterebbe estremamente difficile. Si favorirebbero infatti le stazioni con meno latenza, riducendo l'equità generale del mezzo.

Si introduce inoltre il **ritardo di trasmissione** τ , una misura che quantifica il tempo necessario per l'invio del segnale alle altre stazioni per far sì che leggano il canale occupato. Il segnale non si invia istantaneamente, ed è infatti legato al tempo di vulnerabilità, che in questo contesto ha generalmente valore 2τ .

Algorithm 4 CSMA-persistent()

```

while Stazione ha dati da trasmettere do
  while Canale non è libero do
    Attendi che si liberi
  end while
  Trasmetti la trama
  if Canale è occupato then
    Estrai un tempo casuale
  end if
  while Tempo non scaduto do
    Decrementa il tempo
  end while
end while
  
```

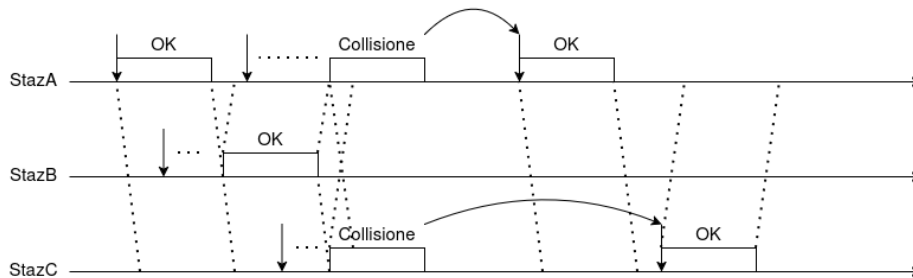


Figura 5.3: Evoluzione di CSMA persistente nel tempo

Un altro approccio per cercare di ridurre il numero di collisioni è il **CSMA non persistente**, una configurazione che non implementa l'attesa del termine della trasmissione in caso di collisione, bensì solo l'estrazione del tempo da attendere per effettuare un nuovo tentativo, con lo scopo di aumentare l'equità tra le stazioni. Riduce il numero di collisioni possibili grazie alla generazione di intervalli di attesa casuali.

Algorithm 5 CSMA-non-persistent()

```

while Stazione ha dati da trasmettere do
  if Canale è libero then
    Trasmetti la trama
  else Estrai un tempo casuale
  end if
  while Tempo non scaduto do
    Decrementa tempo
  end while
end while
  
```

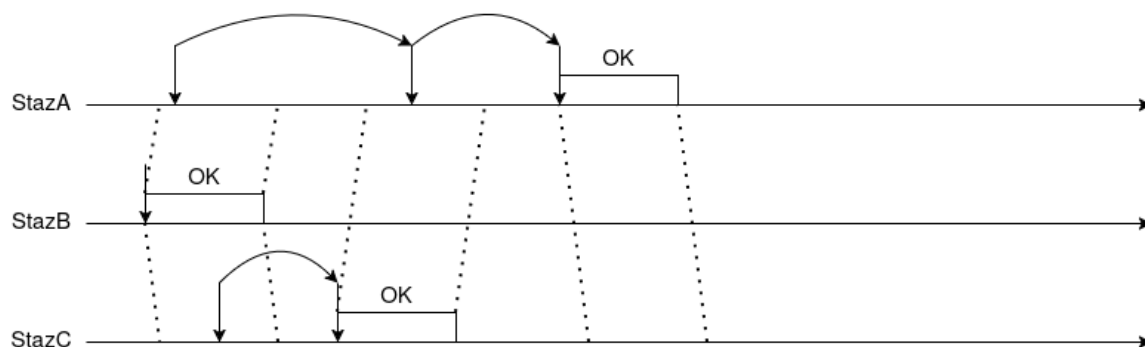


Figura 5.4: Evoluzione di CSMA non persistente nel tempo

Un ulteriore approccio è fornito dal **CSMA-p-persistent**, il quale introduce una variabile p configurabile che indica la probabilità di una stazione di trasmettere la propria trama in un microslot di tempo. Questo implica una riduzione delle collisioni se questi sono stabiliti con una grandezza sufficiente per tenere conto del ritardo di trasmissione, consentendo una "migliore" lettura del canale. È ugualmente possibile collidere se due stazioni trasmettono nello stesso microslot, ma è una probabilità molto bassa.

Algorithm 6 CSMA-p-persistent()

```

while Stazione ha dati da trasmettere do
  if Canale è libero then
    Trasmetti la trama
  else
    while Canale non è libero do
      Trasmetti la trama con probabilità  $p$ 
      Rimanda la trasmissione di un microslot con probabilità  $1 - p$ 
    end while
  end if
  if Collisione verificata then
    Estrai un tempo casuale
  end if
  while Tempo non scaduto do
    Decrementa tempo
  end while
end while
  
```

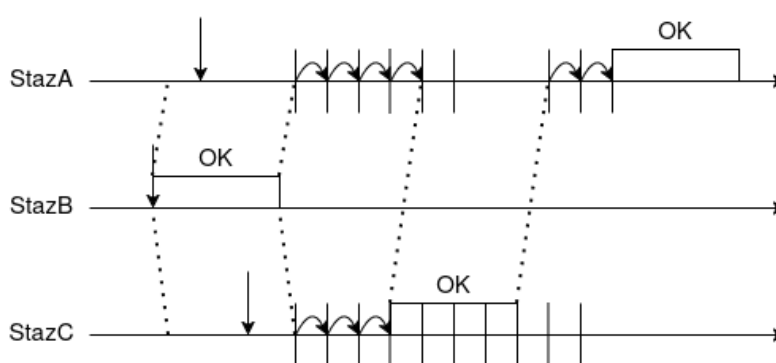


Figura 5.5: Evoluzione di CSMA P-persistente nel tempo

L'ultima sfaccettatura del CSMA, combinabile con tutte le varianti precedenti, è quella di **Collision Detection** (CSMA-CD), resa possibile grazie alle nuove architetture, che permette di interrompere la trasmissione in presenza di collisioni. Presentiamo la dinamica applicata all'algoritmo CSMA persistente.

Algorithm 7 CSMA-CD()

```

while Stazione ha dati da trasmettere do
  while Canale non è libero do
    Attendi che si liberi
  end while
  Trasmetti la trama
  if Canale è occupato then
    Interrompi la trasmissione
    Estrai un tempo casuale
  end if
  while Tempo non scaduto do
    Decrementa il tempo
  end while
end while
  
```

▷ Unica aggiunta, sì.

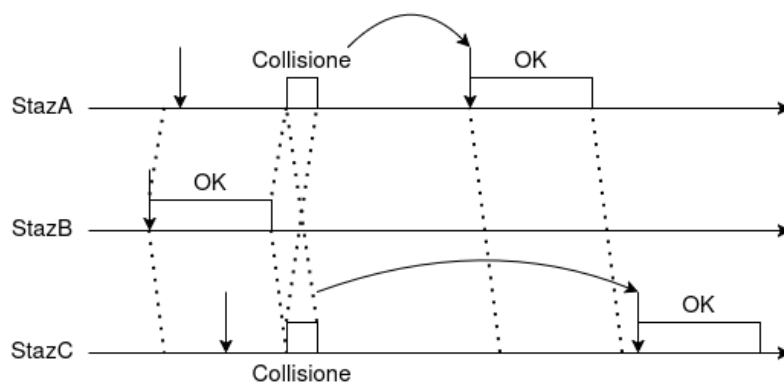


Figura 5.6: Evoluzione di CD-CSMA nel tempo

Grazie alla dinamica di Collision Detection l'efficienza del CSMA aumenta drasticamente, in particolare nel migliore degli scenari fino al 58% per l'algoritmo persistente, mentre fino al 90% per il p-persistente, con $p = 0.01$.

5.3 Ethernet, switch e LAN, Bridge

Il protocollo **Ethernet** è quello attualmente più utilizzato nelle reti LAN cablate; si tratta di una tecnologia nata negli anni '70 le cui prime implementazioni si servivano di un cavo coassiale come bus per connettere ogni nodo. Questa topologia rendeva le reti che ne facevano utilizzo delle **LAN Broadcast**, in quanto tutti i frame trasmessi venivano elaborati da tutte le schede di rete connesse al bus.

Alla fine degli anni '90 si passa ad una topologia a stella, dove host e router sono collegati direttamente a un **hub**, un dispositivo a livello fisico che, ricevuto un bit a un'interfaccia, ne amplifica la potenza e lo trasmette su tutte le altre.

Negli anni 2000 Ethernet subì un'altra trasformazione; dove è rimasta la topologia a stella, l'hub è stato sostituito da uno **switch**, un apparecchio che utilizza i soli livelli di collegamento dati e fisico ed è formato da un insieme di porte alle quali sono collegati i vari host della rete. Svolge funzioni di store-and-forward ed è immune alle collisioni. Le reti che lo implementano sono dette **Switched LAN**, ovvero reti locali commutate.

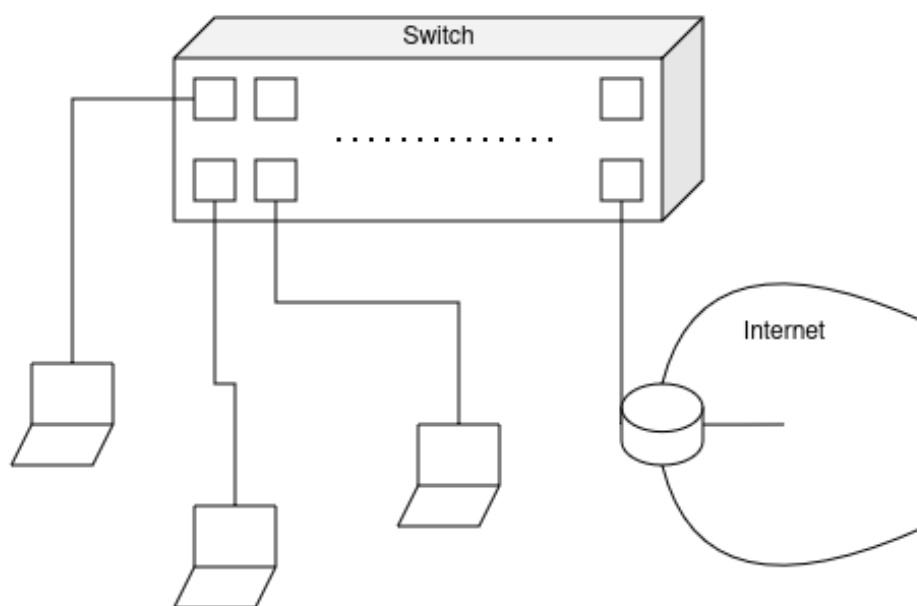


Figura 5.7: Struttura di uno switch

Il pacchetto a livello 2 attraversa lo switch dal canale fisico (la porta, appunto) adeguato per essere trasportato al router, determinato grazie a una tabella in cui è associato l'indirizzo MAC dell'host o del router con la porta alla quale sono connessi. In caso di una consegna diretta, non è richiesto l'utilizzo del router ed è sufficiente lo switch per il trasporto dei pacchetti, mentre per quella indiretta è necessario inviare i dati alla

porta del router, il quale decapsulerà le intestazioni presenti fino a livello di rete, per poi procedere con i consueti algoritmi di instradamento relativi alla consegna.

È possibile fare uso di più switch collegandoli tra di loro; si tratta di una configurazione utilizzata per la gestione di un edificio su più piani, per esempio. Ogni piano presenterà un determinato numero di host, collegati al relativo switch della locazione, i quali potranno vedere i loro indirizzi MAC. Ogni switch del piano è a sua volta collegato a uno **switch radice**, capace di vedere solo le porte degli switch ai quali rigirare i dati, e anche l'unico collegato al router di bordo per consentire agli end-host di interfacciarsi su Internet.

Una configurazione tale presenta un grande vantaggio per gli amministratori di rete: se la radice è connessa solo agli switch dei piani e al router di bordo, non sarà necessario modificarla in base agli host presenti sui piani, perché non può vederli. Tuttavia, è necessario prestare attenzione a non creare dei cicli, perché in tal caso si avrebbero più porte associate ad uno stesso indirizzo MAC.

L'intestazione del protocollo utilizzato, standardizzato da IEEE con codice 802.3, presenta la seguente struttura:

- **Campo dati:** Contiene il datagramma IP la cui dimensione minima dovrà essere $46B$. Se inferiore, si riempie il campo di bit fino a raggiungere tale valore, per poi rimuovere lo "stuffing" grazie al campo di lunghezza header del datagramma. Se invece la dimensione supera la MTU di $1500B$ il pacchetto dovrà essere frammentato.
- **Indirizzo MAC destinazione:** Indica l'indirizzo hardware della scheda di rete di destinazione. Richiede $6B$.
- **Indirizzo MAC sorgente:** Indica l'indirizzo hardware della scheda di rete di sorgente. Richiede $6B$.
- **Tipo:** Consente a Ethernet di supportare più protocolli di rete. Richiede $2B$.
- **Controllo a ridondanza ciclica:** Svolge una funzione di verifica della correttezza dei bit del frame, equivalente al checksum. Richiede $4B$.
- **Preambolo:** Necessario per sincronizzare le stazioni tra di loro grazie ai primi sette byte e a comunicare l'inizio dei dati con gli ultimi due bit dell'ultimo byte. In totale richiede $8B$.

La coesistenza di indirizzi IP e indirizzi MAC tuttavia implica la necessità di un meccanismo per l'accoppiamento dei due, in tal modo che risultino associati alla stessa scheda di rete e interfaccia dell'host. Il protocollo risolutivo è chiamato **Address Resolution Protocol** e svolge una funzione analoga al DNS, ma per gli indirizzi MAC della stessa sottorete. La dinamica segue i passi:

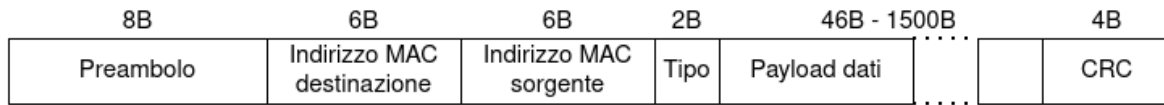


Figura 5.8: Intestazione protocollo Ethernet

1. Se un host non conosce un indirizzo MAC, a partire da un dato indirizzo IP, crea una richiesta ARP (ARP-req), la quale contiene un campo che segnala questo tipo di messaggio e un altro contenente l'indirizzo IP per il quale si richiede il relativo indirizzo MAC. Il pacchetto è incapsulato in un header di livello 2 ed è inviato in broadcast ad ogni host della rete.
2. Ogni host ricevente il pacchetto rimuove l'intestazione di livello 2 e controlla l'IP di destinazione nel payload ricevuto. Se corrisponde, si crea una risposta ARP (ARP-asw) con tre campi dove sono segnalati il tipo di messaggio, l'IP di chi sta rispondendo e il relativo indirizzo MAC. Il tutto è naturalmente incapsulato in un header di livello 2 dove il MAC di destinazione corrisponde a quello che ha fatto la richiesta iniziale e quello di sorgente indica quello di chi sta rispondendo.

Gli accoppiamenti tra indirizzi IP e indirizzi MAC, come anche le ARP-asw ottenute, sono salvati in una **tabella ARP** in cui ogni riga ha un determinato tempo di validità, al termine del quale viene cancellato.

Introduciamo adesso il problema di **delimitazione delle trame**. Richiamando il tempo in cui gli apparecchi switch erano troppo costosi e con un numero di porte limitato rispetto a quelli odierni, era possibile utilizzarne uno solo al quale erano collegati tutti i cavi coassiali della struttura. Ciò creava un contesto nel quale erano presenti almeno due cavi indipendenti connessi allo stesso apparecchio, definito **bridge**. Quando una stazione genera una trama, quest'ultimo la recupera dal segmento (cavo coassiale) al quale è collegata e la memorizza internamente. Se la destinazione si trova sull'altro segmento, procede a trasmettervi la trama.

In quanto cavi indipendenti, se due stazioni trasmettono allo stesso tempo ma sono collegate a segmenti diversi, non si ha una collisione. Inoltre, il bridge utilizzerà il suo buffer di entrata per salvare le trame ricevute e quello di uscita per effettuare tentativi di trasmissione in base all'algoritmo scelto dal segmento.

Assumiamo ora di dover trasmettere più trame di livello 2 consecutivamente. Queste arriveranno al buffer di entrata della porta del bridge e saranno trasmesse una dietro l'altra. Sarà necessario passare dal livello fisico, ma a questo punto bisogna fornire una modalità per distinguere l'inizio e la fine di ogni trama inviata, perché a livello fisico sono modulate e demodulate in base al mezzo scelto.

Si potrebbe pensare di introdurre intervalli tra le trame, ma ha alcuni svantaggi degni di nota, come lo spreco di risorse dovuto a questi spazi di silenzio, oppure la fragilità in caso di distorsione del segnale. Un'altra idea è quella del **character count**, che introduce nell'header di livello 2 un campo che indica il numero di Byte della trama. Problematica anch'essa, però, in quanto ogni trasmissione comporta un potenziale errore a causa delle distorsioni, disallineando di conseguenza il valore della trama distorta come anche quelle successive.

La soluzione più utilizzata è quella del **byte di flag e bit di stuffing**, che vede una specifica sequenza di bit aggiunta all'inizio e alla fine di ogni trama. Usata da sola risulterebbe estremamente fragile, poiché la sequenza potrebbe presentarsi all'interno del campo dei dati, quindi si aggiunge il bit di stuffing; in particolare, in trasmissione, ad ogni sequenza equivalente a quella scelta trovata nei dati si aggiunge uno 0, mentre in ricezione lo si rimuove per riottenere i dati originali della trama.

5.4 Wireless LAN

Introduciamo adesso l'architettura di rete più comunemente utilizzata: le **Wireless LAN** (Wi-Fi). In questo modello è presente un apparecchio supplementare chiamato **access point**, dotato di una stack protocollare di livello fisico e collegamento dati e posto tra il router e gli end-host. Svolge una funzione equivalente a quella del bridge ed è connesso tramite due interfacce agli host e al router rispettivamente.

Questi apparecchi sono mezzi la cui capacità è condivisa con tutti gli host di una determinata rete; ciò implica la presenza di collisioni, ed è necessario fornire una nuova variante del CSMA per una gestione quanto più ottimale. Questo protocollo, chiamato **CSMA-CA** (Collision Avoidance) non prevede una comunicazione diretta tra gli host¹, forzandoli quindi ad appellarsi all'access point per comunicazioni dirette e indirette.

Il mezzo condiviso agisce quindi da manager e ha anche il compito di gestire la temporizzazione per allineare correttamente gli host coinvolti; in particolare, il tempo disponibile è diviso in vari slot la cui dimensione è determinata dal tempo di propagazione tra access point e la stazione più distante, rendendo il valore dipendente dalla tecnologia implementata. In genere, gli slot hanno un intervallo di dimensione molto piccola, come $10\mu s$, se consideriamo IEEE 802.116. Definiamo quindi due tipi di intervallo per il CSMA-CA:

- **SIFS**: Short InterFrame Space, equivalente alla dimensione minima dello slot, quindi a τ .
- **DIFS**: Distributed InterFrame Space, equivalente a tre volte l'intervallo base, quindi a 3τ .

¹Il discorso vale per una configurazione normale, ma esiste anche una variante che consente di comunicare direttamente in determinati contesti.

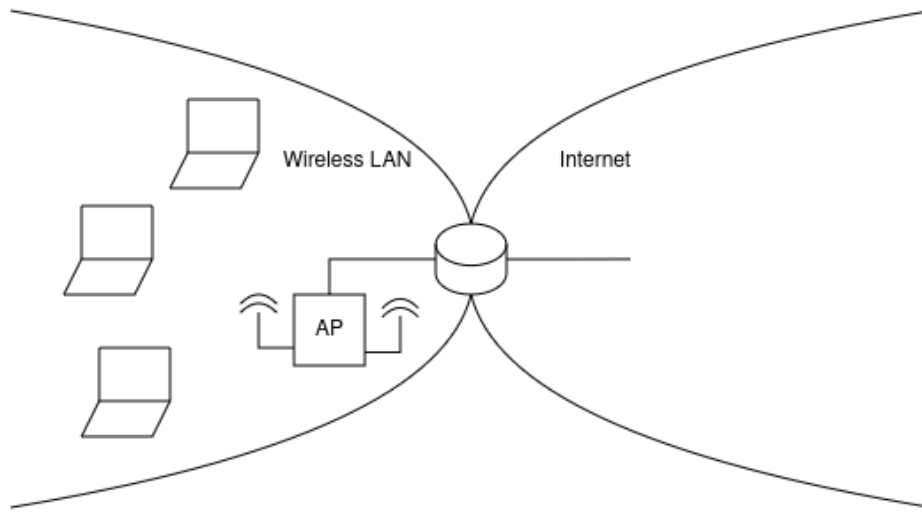


Figura 5.9: Struttura rete wireless

Tutti gli host sono sincronizzati dall'access point per quanto riguarda la temporizzazione, di conseguenza hanno tutti presente quando inizia un nuovo slot. La procedura richiede inoltre due valori specifici: cw , detto **contention window**, indica il numero di slot totali che si possono potenzialmente attendere, mentre s è il numero di slot SIFS da attendere prima di poter trasmettere. Presentiamo l'algoritmo con un esempio in quanto nettamente più complesso rispetto ai precedenti:

1. Supponiamo un access point che collega due segmenti. Il primo contiene la stazione A, mentre il secondo presenta B e C.
2. L'AP trasmette una trama t_{AP} con successo, e durante questo intervallo di tempo così generano anche A e B le proprie t_A, t_B . Vedendo tuttavia il canale occupato, continueranno ad ascoltare il canale fino alla sua liberazione.
3. Una volta liberato il canale, A e B continuano ad ascoltarlo per un intervallo DIFS, al termine del quale estrarranno il proprio valore casuale s . Supponiamo $s_A = 6$ e $s_B = 10$.
4. Si decrementano quindi i due valori e dato che A presentava quello minore, raggiungerà lo 0 per prima e potrà quindi trasmettere la sua trama. Vedendo il canale nuovamente occupato, B salva il suo counter, arrivato a $s_B = 4$ e ritorna ad attendere la liberazione del mezzo. Inoltre, durante la trasmissione di t_A , C ha generato la propria trama t_C , che come t_B dovrà attendere il termine di t_A .
5. Conclusa la trasmissione della trama di A, B e C attenderanno il DIFS per poi riprendere il conteggio interrotto nel caso di B ed estrarre un s nel caso di C. Supponiamo $s_C = 8$.

6. Il valore di s_B è il minore, quindi B potrà trasmettere la sua trama quando raggiungerà lo zero, mentre s_C arriva a 4 e salverà il proprio valore.
7. Terminata la trasmissione di t_B , C potrà attendere il DIFS e decrementare il suo s_C per trasmettere finalmente la sua trama, terminando la comunicazione in toto.

Algorithm 8 CSMA-CA

```

while Stazione ha dati da trasmettere do
  Verifica stato del canale
  if Canale libero then
    Ascoltalo ancora per un tempo pari a un DIFS
    Se rimane libero al termine dell'attesa, trasmetti.           ▷ Terminazione
  else
    while Canale occupato do
      Attendi la sua liberazione
    end while
  end if
  Verifica stato del canale
  if Canale rimane libero per un intervallo DIFS then
    Se non esiste, estrai un numero uniformemente distribuito  $s$  tra  $(0, cw - 1)$ 
    while Canale è ancora libero do
       $s \leftarrow s - 1$ 
      if  $s = 0$  then
        Trasmetti la trama                                     ▷ Terminazione
      end if
    end while           ▷ Se esce con  $s \neq 0$ , il canale è di nuovo occupato
    Salva il valore corrente di  $s$ 
  else if Collisione verificata then
    Interrompi la trasmissione
    Estrai un tempo casuale da attendere e raddoppia  $cw$ 
  end if
end while

```

Ne risulta evidente come il protocollo favorisca stazioni fortunate abbastanza da estrarre valori s più piccoli rispetto alle altre. Nonostante ciò, è esattamente questa casualità a renderlo equo rispetto a tutte.

Ma a questo punto, qual è lo scopo del DIFS se è necessario attendere ugualmente un numero di s slot? Ebbene, nelle reti wireless è possibile attivare un riscontro di livello 2, quindi una ricevuta di consegna da parte dell'access point in merito alla trama; relativo, naturalmente, al singolo hop tra host e access point, ed è inviato subito dopo un SIFS

dalla fine della trasmissione della relativa trama. Il DIFS è necessario per garantire tempo a sufficienza per la trasmissione di questo acknowledgement.

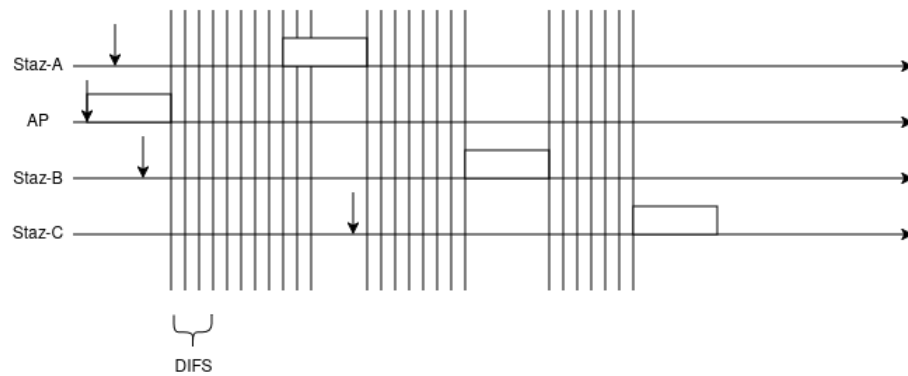


Figura 5.10: Algoritmo CSMA-CA

5.5 Esercizi svolti